

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM

KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC

**ĐỀ TÀI:**

**Xây dựng CI/CD pipeline bảo mật (DevSecOps) - tích hợp  
DAST, IaC scanning, container image scanning**

**Môn học:** NT524.Q21.ANTT - Kiến trúc và Bảo mật Điện toán Đám mây

**Giảng viên hướng dẫn:** Nguyễn Ngọc Tụ

**Thực hiện bởi nhóm 05, bao gồm:**

- |                   |          |             |
|-------------------|----------|-------------|
| 1. Huỳnh Anh Khôi | 23520764 | Trưởng nhóm |
| 2. Lê Nhật Trường | 23521691 | Thành viên  |

**Thời gian thực hiện:** 3 tháng

## **Mục lục**

|                                                          |    |
|----------------------------------------------------------|----|
| DANH SÁCH HÌNH, BẢNG.....                                | 6  |
| DANH MỤC TỪ TẠM DỊCH.....                                | 7  |
| DANH MỤC TỪ VIẾT TẮT .....                               | 8  |
| TÓM TẮT.....                                             | 9  |
| Chương I. TỔNG QUAN. ....                                | 11 |
| 1. Giới thiệu đề tài .....                               | 11 |
| 1.1. Tên đề tài .....                                    | 11 |
| 1.2. Đặt vấn đề.....                                     | 11 |
| 1.3. Lý do chọn đề tài .....                             | 12 |
| 1.4. Mục tiêu của đề tài.....                            | 12 |
| 1.5. Cơ sở lý thuyết.....                                | 13 |
| 1.5.1. DevSecOps và Pipeline CI/CD .....                 | 13 |
| 1.5.2. Tư duy Shift-Left.....                            | 15 |
| 1.5.3. Cloud computing .....                             | 15 |
| 1.5.4. Môi trường Staging và môi trường Production ..... | 16 |
| 1.5.5. Hệ thống giám sát tập trung.....                  | 17 |
| 1.5.6. Secrets và Secret scanning.....                   | 17 |
| 1.5.7. Dependency scanning .....                         | 17 |
| 1.5.8. IaC và IaC scanning.....                          | 18 |
| 1.5.9. Container image và SBOM:.....                     | 18 |
| 1.5.10. DAST.....                                        | 19 |
| Chương II. THIẾT KẾ KIẾN TRÚC.....                       | 21 |
| 1. Mô hình hóa các mối đe dọa.....                       | 21 |

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| 2. Kiến trúc tổng quan .....                                            | 22 |
| 2.1. Giai đoạn đánh giá sơ bộ .....                                     | 23 |
| 2.2. Giai đoạn CI.....                                                  | 23 |
| 2.3. Giai đoạn CD .....                                                 | 23 |
| 2.4. Hệ thống lưu trữ tập trung: .....                                  | 24 |
| Chương III: Triển khai .....                                            | 25 |
| 1. Công nghệ sử dụng .....                                              | 25 |
| 1.1. GitHub và GitHub Actions .....                                     | 25 |
| 1.2. Gitleaks .....                                                     | 26 |
| 1.3. Semgrep .....                                                      | 26 |
| 1.4. TruffleHog .....                                                   | 26 |
| 1.5. tfsec .....                                                        | 27 |
| 1.6. Dependabot.....                                                    | 27 |
| 1.6.1. Terraform .....                                                  | 28 |
| 1.7. Kubernetes .....                                                   | 28 |
| 1.8. Docker.....                                                        | 28 |
| 1.9. Checkov .....                                                      | 29 |
| 1.10. Confest.....                                                      | 29 |
| 1.11. Trivy.....                                                        | 30 |
| 1.12. Cosign .....                                                      | 30 |
| 1.13. Trực quan hóa dữ liệu - <i>Grafana</i> .....                      | 31 |
| 1.14. Hệ thống thu thập chỉ số định lượng - <i>Prometheus</i> .....     | 31 |
| 1.15. Hệ thống quản lý và phân tích nhật ký sự kiện - Grafana Loki..... | 32 |
| 1.16. OWASP ZAP .....                                                   | 32 |

|                                                |    |
|------------------------------------------------|----|
| 1.17. OPA Gatekeeper.....                      | 32 |
| 1.18. AWS.....                                 | 33 |
| 1.18.1. AWS ECR.....                           | 33 |
| 1.18.2. AWS EKS.....                           | 34 |
| 2. Chi tiết triển khai.....                    | 34 |
| 2.1. Workflow.....                             | 34 |
| 2.2. Hệ thống thu thập dữ liệu tập trung:..... | 36 |
| 2.3. Ứng dụng web mẫu.....                     | 37 |
| 2.4. IaC bằng Terraform/hashicorp.....         | 38 |
| 2.5. Pre-commit.....                           | 40 |
| 2.6. Secret scanning:.....                     | 44 |
| 2.7. Dependency scanning.....                  | 45 |
| 2.8. IaC scanning.....                         | 46 |
| 2.9. Buid và Container scanning.....           | 47 |
| 2.10. Triển khai hạ tầng.....                  | 49 |
| 2.11. Triển khai ứng dụng.....                 | 51 |
| 2.12. DAST.....                                | 54 |
| 2.13. Policy Gate.....                         | 56 |
| Chương IV: KẾT QUẢ TRIỂN KHAI.....             | 58 |
| 1. Pre-commit.....                             | 58 |
| 2. Secret scanning.....                        | 59 |
| 3. IaC scanning:.....                          | 62 |
| 4. Dependency scanning.....                    | 64 |
| 5. Container scanning.....                     | 64 |

|                                                                  |    |
|------------------------------------------------------------------|----|
| 6. DAST.....                                                     | 66 |
| 7. Grafana Prometheus.....                                       | 69 |
| 8. Grafana Loki.....                                             | 72 |
| 9. Sản phẩm thực tế: .....                                       | 74 |
| Chương V. KẾT LUẬN. ....                                         | 75 |
| 1. Kết quả đạt được của đồ án .....                              | 75 |
| 2. Những hạn chế và sai sót gặp phải .....                       | 75 |
| 3. Hướng phát triển và giải pháp khắc phục trong tương lai ..... | 75 |
| Tài liệu tham khảo .....                                         | 77 |

## **DANH SÁCH HÌNH, BẢNG**

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| Hình 1: Tổng quan về DevSecOps .....                                      | 14 |
| Hình 2: Kiến trúc tổng quan .....                                         | 22 |
| Hình 3: Kiến trúc triển khai .....                                        | 25 |
| Hình 4: Các workflow trong Quy trình.....                                 | 35 |
| <br>                                                                      |    |
| Bảng 1: Cơ chế gọi các workflow khác .....                                | 35 |
| Bảng 2: Lệnh chuyển đổi payload thành JSON .....                          | 37 |
| Bảng 3: Cấu hình workflow để dùng pre-commit .....                        | 41 |
| Bảng 4: Cấu hình pre-commit trong file .pre-commit-config.yaml.....       | 42 |
| Bảng 5: Cấu hình chính trong Secret scanning.....                         | 44 |
| Bảng 6: Lệnh kết nối đến cluster .....                                    | 52 |
| Bảng 7: Nội dung cấu hình trong file app-deploy.yaml.....                 | 53 |
| Bảng 8: Các rule được cấu hình.....                                       | 55 |
| Bảng 9: Bảng tổng hợp lỗi được quét ở pre-commit .....                    | 59 |
| Bảng 10: Bảng thống kê chi tiết các cảnh báo ở bước Secret scanning ..... | 61 |
| Bảng 11: Bảng tổng hợp cảnh báo từ bước IaC scanning.....                 | 62 |
| Bảng 12: bảng tổng hợp kết quả ở bước container scanning.....             | 66 |
| Bảng 13: Bảng tổng hợp kết quả ở bước DAST .....                          | 68 |

**DANH MỤC TỪ TẠM DỊCH**

| <b>Từ tiếng anh</b>     | <b>Từ tiếng việt</b>                 |
|-------------------------|--------------------------------------|
| Secrets                 | Thông tin nhạy cảm                   |
| Misconfiguration        | Cấu hình sai lệch                    |
| Container Image         | Ảnh đóng gói máy ảo                  |
| Pipeline                | Đường ống (hoặc quy trình)           |
| Fast Feedback           | Phản hồi nhanh                       |
| Dependency              | Quét thư viện (thành phần phụ thuộc) |
| Scanning                | Quét                                 |
| Staging environment     | Môi trường giả lập                   |
| Policy as Code          | Chính sách bảo mật dạng mã nguồn     |
| Cloud Security Engineer | Kỹ sư bảo mật đám mây                |
| Policy Gate             | Chốt chặn chính sách                 |
| Repository              | Kho lưu trữ                          |
| Ticket                  | Thẻ công việc                        |

**DANH MỤC TỪ VIẾT TẮT**

| <b>Tên viết tắt</b> | <b>Tên đầy đủ</b>                                       |
|---------------------|---------------------------------------------------------|
| SAST                | Static Application Security Testing                     |
| IaC                 | Infrastructure as Code                                  |
| DevSecOps           | Development, Security, and Operations                   |
| CI/CD               | Continuous Integration & Continuous Delivery/Deployment |
| PR                  | Pull Request                                            |
| CVE                 | Common Vulnerabilities and Exposures                    |
| DAST                | Dynamic Application Security Testing                    |
| SBOM                | Software Bill of Materials                              |
| SDLC                | Software Development Life Cycle                         |
| QA                  | Quality Assurance                                       |

## TÓM TẮT

Trong kỷ nguyên chuyển đổi số và xu hướng phát triển phần mềm theo mô hình Agile/DevOps diễn ra mạnh mẽ, tốc độ bàn giao sản phẩm liên tục được đẩy lên rất cao, song điều này cũng vô tình tạo ra những kẽ hở lớn về an toàn thông tin khi mã nguồn và hạ tầng được đẩy lên môi trường vận hành mà thiếu đi sự kiểm duyệt kỹ lưỡng. Các sự cố nghiêm trọng như mã nguồn chứa secrets, tồn tại lỗ hổng trong các thư viện phụ thuộc của bên thứ ba, IaC misconfigurations, hay mã độc cài cắm trong Container Image ngày càng trở nên phổ biến và tinh vi.

Xuất phát từ thực trạng đó, đồ án này tập trung nghiên cứu và xây dựng một hệ thống DevSecOps CI/CD Pipeline tự động hóa hoàn chỉnh dựa trên triết lý "Shift-Left", nhằm đưa việc kiểm thử bảo mật vào sớm nhất có thể trong vòng đời phát triển phần mềm. Hệ thống này không chỉ tự động ngăn chặn các thay đổi kém an toàn thông tin thông qua các rào chắn chính sách, mà còn tối ưu hóa trải nghiệm của lập trình viên nhờ vào cơ chế Fast Feedback ngay trong quá trình phát triển mã nguồn. Pipeline được thiết kế vận hành khép kín và tự động hóa theo vòng đời nghiêm ngặt từ giai đoạn pre-commit, thiết lập PR, đóng gói, kiểm thử cho đến khi phân phối sản phẩm. Trong chu trình này, đồ án đã tích hợp thành công một loạt giải pháp kiểm thử bảo mật toàn diện xuyên suốt các lớp ứng dụng và hạ tầng. Quy trình bao gồm: Secret Scanning, Dependency Scanning, IaC scanning, Container Image Scanning trước khi lưu trữ. Đặc biệt, khi ứng dụng được triển khai lên môi trường Staging, hệ thống tiếp tục kích hoạt tự động DAST.

Điểm cốt lõi của hệ thống nằm ở giai đoạn quản lý an toàn chuỗi cung ứng phần mềm và thiết lập chốt chặn chính sách nghiêm ngặt (Policy-gating). Hệ thống tự động khởi tạo, xác thực SBOM thông qua công cụ chuyên dụng, đồng thời tiến hành ký số xác thực Container Image. Tiếp đó, các chính sách (Policy as Code) được thực thi nghiêm ngặt. Kết quả nghiên cứu của đồ án đã xây dựng thành công Pipeline CI/CD cung cấp luồng xử lý lỗi (triage flow) minh bạch và hệ thống báo cáo trực quan cho các

bên liên quan. Đây không chỉ là giải pháp tối ưu cho các doanh nghiệp đang có nhu cầu chuyển dịch sang mô hình DevSecOps, mà còn là tài liệu nghiên cứu thực tế có giá trị cao cho lộ trình phát triển năng lực của một Cloud Security Engineer chuyên nghiệp.

Cấu trúc bài báo cáo gồm:

- Chương I: TỔNG QUAN
- Chương II: THIẾT KẾ KIẾN TRÚC
- Chương III: TRIỂN KHAI
- Chương IV: KẾT QUẢ TRIỂN KHAI
- Chương V: KẾT LUẬN

## Chương I. TỔNG QUAN.

### 1. Giới thiệu đề tài

#### 1.1. Tên đề tài

**Tên tiếng Việt:** Xây dựng CI/CD pipeline bảo mật (DevSecOps) - tích hợp DAST, IaC scanning, container image scanning.

**Tên tiếng Anh:** Building CI/CD pipeline security (DevSecOps) - integrating DAST, IaC scanning, container image scanning.

#### 1.2. Đặt vấn đề

Trong bối cảnh công nghiệp phần mềm hiện đại, việc áp dụng các mô hình phát triển linh hoạt (Agile/DevOps) cùng hệ thống tích hợp và triển khai liên tục (CI/CD) đã giúp các tổ chức tối ưu hóa tốc độ bàn giao sản phẩm lên mức chưa từng có. Tuy nhiên, tốc độ phát hành nhanh chóng này lại vô tình tạo ra những kẽ hở lớn về an toàn thông tin nếu thiếu đi sự kiểm duyệt kỹ lưỡng. Trên thực tế, các sai sót bảo mật rất dễ dàng "lọt" qua các đường ống CI/CD truyền thống để xâm nhập trực tiếp vào môi trường vận hành (Production). Các mối đe dọa phổ biến và nguy hiểm nhất thường trực bao gồm: việc lộ lọt thông tin nhạy cảm (secrets/credentials/API keys) do bị cố tình đưa vào trong mã nguồn, rủi ro từ lỗ hổng chưa được vá trong các thư viện phụ thuộc của bên thứ ba, sai sót nghiêm trọng trong quá trình khai báo hạ tầng mã nguồn, và việc sử dụng các container image không đảm bảo an toàn, dễ chứa mã độc.

Để đối phó với những thách thức trên, các tổ chức và doanh nghiệp đang đặt ra yêu cầu cấp thiết về việc nâng cấp hệ thống CI/CD truyền thống thành các DevSecOps pipeline. Hệ thống này cần đóng vai trò như những "chốt chặn" tự động, có khả năng phát hiện và chặn đứng các thay đổi mã nguồn kém an toàn trước khi chúng được sáp nhập hoặc triển khai. Mặc dù vậy, việc áp đặt các quy định bảo mật khắt khe thường dễ dẫn đến tình trạng ách tắc, làm chậm tiến độ dự án. Do đó, bài toán cốt lõi đặt ra là hệ thống tự động hóa này phải duy trì được trải nghiệm tối ưu cho đội ngũ phát triển thông qua cơ chế phản hồi nhanh, giúp nhận biết và khắc phục lỗi ngay tại thời điểm viết code theo đúng triết lý "Shift-Left".

Mặc dù nhu cầu tuyển dụng và ứng dụng thực tiễn của doanh nghiệp đối với hệ thống DevSecOps là rất lớn, việc tiếp cận và thực hành xây dựng một hệ thống bảo mật toàn diện từ con số không vẫn là một thách thức đối với sinh viên và những người mới bắt đầu. Xuất phát từ thực trạng đó, đề tài này được thực hiện với mục tiêu xây dựng một hệ thống DevSecOps pipeline khép kín hoàn chỉnh. Đây được xem là nền tảng kiến thức và kỹ năng thực chiến quan trọng, giúp chuẩn bị hành trang vững chắc cho định hướng nghề nghiệp chuyên sâu vào các vị trí Kỹ sư Bảo mật Đám mây hoặc Kỹ sư DevSecOps trong tương lai.

### **1.3. Lý do chọn đề tài**

Quyết định triển khai đề tài xuất phát từ bối cảnh an toàn không gian mạng đang chuyển biến vô cùng phức tạp, đặc biệt là sự gia tăng đột biến của các cuộc tấn công vào chuỗi cung ứng phần mềm và rủi ro từ môi trường điện toán đám mây. Theo báo cáo Cảnh quan Mối đe dọa của Cơ quan An ninh Mạng Liên minh Châu Âu (ENISA), các cuộc tấn công vào chuỗi cung ứng đã gia tăng theo cấp số nhân, trong đó khoảng 66% các cuộc tấn công tập trung vào việc khai thác mã nguồn của nhà cung cấp để xâm nhập vào hệ thống của khách hàng đích [1]. Các tác nhân đe dọa ngày càng nhắm mục tiêu vào các kho lưu trữ mã nguồn mở phổ biến (như npm, PyPI) để chèn mã độc, biến CI/CD pipeline truyền thống thành trung gian phát tán rủi ro [2].

Những số liệu trên minh chứng rõ ràng rằng, hệ thống CI/CD truyền thống nếu thiếu đi các chốt chặn tự động sẽ trở thành "cửa ngõ" để rủi ro xâm nhập thẳng vào môi trường Production. Các tổ chức hiện nay đang cực kỳ khát khao một hệ thống DevSecOps hoàn thiện, vừa có khả năng đánh chặn các đoạn mã độc hại, vừa duy trì được trải nghiệm tốc độ phản hồi nhanh (fast feedback) để không cản trở luồng công việc của kỹ sư phần mềm. Do đó, việc nghiên cứu và áp dụng mô hình tự động hóa DevSecOps là một nhu cầu thực tiễn và vô cùng cấp thiết [3].

### **1.4. Mục tiêu của đề tài**

Thiết kế và xây dựng repository mẫu hoàn chỉnh, bao gồm mã nguồn của một ứng dụng web cơ bản, các tệp lệnh đóng gói Dockerfile và IaC. Triển khai một hệ thống Pipeline CI/CD tích hợp các chơ chế như kiểm tra mã nguồn nhanh (pre-commit

checks), quét thông tin mật, quét thành phần phụ thuộc, và đánh giá an toàn hạ tầng, tự động đóng gói container và quét bảo mật, khởi tạo danh mục thành phần phần mềm (SBOM), ký xác thực image (được đóng gói ở bước trước) và thực hiện kiểm thử bảo mật động (DAST) trên môi trường giả lập (Staging).

Định nghĩa chốt chặn chính sách và thực thi các chính sách một cách tự động và nghiêm ngặt. Hệ thống sẽ được thiết lập các ngưỡng rủi ro bảo mật) nhằm tự động ngăn chặn PR nếu phát hiện các nguy cơ vượt quá mức cho phép. Đối với một số lỗi cấu hình IaC thông thường, hệ thống được cấu hình để có khả năng tự động tạo các bản vá trong những điều kiện an toàn. Tích hợp quy trình phân loại và xử lý sự cố một cách liền mạch bằng việc tự động khởi tạo các ticket trên JIRA hoặc tạo các báo cáo lỗi kèm theo những hướng dẫn, gợi ý khắc phục chi tiết để hỗ trợ tối đa cho đội ngũ phát triển.

Đo lường tính hiệu quả và khả năng áp dụng thực tiễn của hệ thống DevSecOps vừa xây dựng, đề tài đặt mục tiêu tiến hành đánh giá toàn diện dựa trên các chỉ số hiệu suất và bảo mật cốt lõi. Các tiêu chí đánh giá bao gồm: độ trễ của pipeline nhằm đảm bảo tốc độ phản hồi nhanh chóng, tỷ lệ cảnh báo sai để tối ưu hóa độ tin cậy của các bộ công cụ quét, và mức độ cản trở hay ảnh hưởng đến trải nghiệm của nhà phát triển. Đặc biệt, sự cải thiện về mặt an toàn thông tin của toàn bộ dự án sẽ được chứng minh thông qua việc đo lường mức độ suy giảm các lỗ hổng bảo mật được phát hiện và xử lý trước khi ứng dụng được triển khai lên môi trường vận hành thực tế.

## **1.5. Cơ sở lý thuyết**

### **1.5.1. DevSecOps và Pipeline CI/CD**

DevOps (viết tắt của Development và Operations) là sự kết hợp giữa thực hành và bộ công cụ tự động hóa. Mục tiêu cốt lõi của mô hình này là thu hẹp khoảng cách giữa nhóm phát triển phần mềm và nhóm vận hành hệ thống, từ đó đẩy nhanh tốc độ triển khai, cải thiện tính ổn định của sản phẩm và tăng cường khả năng phân phối dịch vụ ở nhịp độ nhanh hơn so với quy trình truyền thống [4] [5] [6] [7].

DevSecOps (Development - Security - Operations) là phương pháp hiện đại, mở rộng từ DevOps bằng cách tích hợp chủ động các quy trình và công cụ bảo mật vào toàn bộ vòng đời phát triển phần mềm (SDLC), thay vì chỉ kiểm tra ở giai đoạn cuối. Mô

hình này đề cao văn hóa "bảo mật là trách nhiệm chung", giúp phát hiện sớm các lỗ hổng, giảm thiểu rủi ro, tự động hóa quy trình an ninh mạng trong CI/CD pipeline, từ đó đảm bảo sản phẩm được bàn giao nhanh chóng nhưng vẫn tuyệt đối an toàn và ổn định [8] [9].



Hình 1: Tổng quan về DevSecOps

CI/CD Pipeline là một quy trình tự động hóa chuỗi các bước phát triển phần mềm bao gồm tích hợp (CI), kiểm thử và triển khai (CD), giúp chuyển đổi mã nguồn từ máy lập trình viên đến môi trường sản xuất một cách nhanh chóng, ổn định và an toàn. Cụ thể, khi lập trình viên commit mã mới, hệ thống sẽ tự động chạy các công đoạn build<sup>1</sup>, unit test, và kiểm tra code style, qua đó phát hiện lỗi sớm và giảm thiểu sự can thiệp thủ công. Nhờ sự kết hợp giữa tích hợp liên tục và phân phối/triển khai liên tục, quy trình này giúp tối ưu hóa hiệu quả làm việc, giảm thời gian phát hành sản phẩm và đảm bảo chất lượng phần mềm nhất quán, trở thành nền tảng cốt lõi trong văn hóa DevOps hiện đại [10] [11].

<sup>1</sup> Build: công đoạn đóng gói ứng dụng

### 1.5.2. Tư duy Shift-Left

Tư duy Shift-Left trong phát triển phần mềm là phương pháp tiếp cận chủ động, trong đó các hoạt động như kiểm thử, đánh giá chất lượng và bảo mật được thực hiện càng sớm càng tốt, ngay từ giai đoạn thu thập yêu cầu và thiết kế. Khác với mô hình truyền thống (thường để kiểm thử ở cuối quy trình), Shift-Left tích hợp kiểm thử xuyên suốt vòng đời phát triển. Bằng cách khuyến khích đội ngũ (bao gồm cả lập trình viên và người kiểm thử) cộng tác chặt chẽ từ ngày đầu tiên, tư duy này giúp phát hiện lỗi từ trong trứng nước, giảm thiểu chi phí sửa lỗi tốn kém ở giai đoạn sau và đẩy nhanh quá trình đưa sản phẩm ra thị trường [12] [13] [14].

### 1.5.3. Cloud computing

Cloud<sup>2</sup> và Cloud Computing<sup>3</sup> đại diện cho một sự thay đổi căn bản trong cách lưu trữ và xử lý dữ liệu, chuyển từ việc sở hữu hạ tầng vật lý tại chỗ sang việc thuê tài nguyên qua Internet. Thay vì phải đầu tư, lắp đặt và bảo trì các máy chủ vật lý, người dùng có thể truy cập các tài nguyên CNTT như sức mạnh tính toán, lưu trữ, và cơ sở dữ liệu trên cơ sở nhu cầu từ các nhà cung cấp dịch vụ. Điều này mang lại lợi ích to lớn như tiết kiệm chi phí ban đầu, khả năng mở rộng linh hoạt, triển khai nhanh chóng và hỗ trợ tối ưu hóa quy trình làm việc cho các doanh nghiệp. Cloud Computing (Điện toán đám mây) hoạt động dựa trên nguyên tắc "chia sẻ tài nguyên", cho phép cung cấp dịch vụ công nghệ theo mô hình trả tiền theo mức sử dụng (pay-as-you-go). Các mô hình dịch vụ đám mây chính bao gồm IaaS (cơ sở hạ tầng), PaaS (nền tảng) và SaaS (phần mềm), giúp doanh nghiệp dễ dàng truy cập vào các công nghệ tiên tiến như trí tuệ nhân tạo, máy học và phân tích dữ liệu lớn mà không cần quản lý hạ tầng phức tạp. Nhờ đó, doanh nghiệp có thể tập trung vào các hoạt động cốt lõi, thay vì dành thời gian và nguồn lực cho việc quản lý phần cứng hay trung tâm dữ liệu [15] [16] [17].

---

<sup>2</sup> Cloud: đám mây

<sup>3</sup> Cloud Computing: điện toán đám mây

#### 1.5.4. Môi trường Staging và môi trường Production

Production environment<sup>4</sup> là nơi ứng dụng/website được vận hành chính thức, phục vụ người dùng cuối thật với dữ liệu thật và lưu lượng truy cập thực tế. Đây là môi trường quan trọng nhất, đòi hỏi sự ổn định cao, độ tin cậy, bảo mật chặt chẽ và khả năng mở rộng để đảm bảo dịch vụ không bị gián đoạn. Bất kỳ thay đổi nào từ staging được đẩy lên production đều phải thông qua quy trình kiểm soát khắt khe, vì bất kỳ lỗi nhỏ nào tại đây cũng có thể ảnh hưởng trực tiếp đến trải nghiệm khách hàng và uy tín của doanh nghiệp [18] [19] [20] [21].

Staging environment là một hệ thống thử nghiệm được thiết lập sao cho gần giống nhất có thể với môi trường thật, bao gồm phần cứng, phần mềm và cấu hình, nhưng hoạt động hoàn toàn tách biệt. Vai trò chính của nó là đóng vai trò như một "bản nháp" cuối cùng để đội ngũ phát triển và QA<sup>5</sup> thực hiện các bước kiểm tra, test tính năng mới, kiểm thử tích hợp và kiểm thử hiệu năng trước khi chính thức đưa lên hệ thống thật. Việc sử dụng staging giúp phát hiện sớm các lỗi nghiêm trọng, từ đó giảm thiểu tối đa rủi ro gây gián đoạn dịch vụ hoặc làm hỏng dữ liệu thực của người dùng [22] [23] [24] [25].

Trong phạm vi đề tài này, môi trường Staging được sử dụng như một môi trường kiểm thử trung gian trước khi triển khai lên production. Sau khi container image được xây dựng thành công và vượt qua các bước kiểm tra bảo mật trong CI pipeline, hệ thống sẽ tự động triển khai ứng dụng lên staging để thực hiện kiểm thử động thông qua công cụ DAST (OWASP ZAP). Việc sử dụng staging giúp mô phỏng điều kiện vận hành thực tế, hỗ trợ phát hiện các lỗi hoặc lỗ hổng bảo mật còn tồn tại trước khi đưa ứng dụng vào sử dụng chính thức.

Môi trường Production là đích cuối cùng của quy trình CI/CD trong đề tài. Việc triển khai lên production chỉ được thực hiện khi ứng dụng đã vượt qua toàn bộ các bước kiểm tra bảo mật như secret scanning, dependency scanning, container scanning, image

---

<sup>4</sup> Production environmen: môi trường thật

<sup>5</sup> QA: Đội ngũ kiểm thử

verification và policy gate. Điều này đảm bảo chỉ những phiên bản ứng dụng an toàn và đáng tin cậy mới được phép cung cấp cho người dùng cuối.

#### **1.5.5. Hệ thống giám sát tập trung**

Hệ thống giám sát tập trung trong DevSecOps là nền tảng cốt lõi kết hợp giữa DevOps và an ninh mạng, đóng vai trò thu thập, tổng hợp và phân tích toàn diện dữ liệu log, metric và sự kiện bảo mật từ mã nguồn, pipeline CI/CD, đến hạ tầng đám mây theo thời gian thực. Giải pháp này phá vỡ các rào cản phòng ban bằng cách cung cấp một cái nhìn trực quan duy nhất cho cả đội ngũ phát triển, vận hành và bảo mật. Nó không chỉ giúp phát hiện sớm các lỗ hổng trước khi triển khai, tự động hóa cảnh báo mà còn theo dõi các hành vi bất thường, từ đó tối ưu hóa tốc độ phản ứng sự cố và đảm bảo tuân thủ các tiêu chuẩn an toàn thông tin khắt khe nhất [26] [27] [28].

#### **1.5.6. Secrets và Secret scanning**

Secret (bí mật) trong phát triển phần mềm và DevOps là các thông tin nhạy cảm được sử dụng để xác thực, phân quyền hoặc mã hóa, bao gồm khóa API, mật khẩu cơ sở dữ liệu, chứng chỉ SSH, và các token truy cập. Việc lộ các bí mật này - thường xảy ra khi lập trình viên vô tình chèn chúng vào mã nguồn, file cấu hình, hoặc nhật ký hệ thống - tạo ra rủi ro bảo mật nghiêm trọng, cho phép kẻ tấn công chiếm đoạt quyền truy cập tài nguyên, đánh cắp dữ liệu hoặc tấn công hệ thống [29] [30].

Secret scanning (quét bí mật) là quy trình bảo mật tự động rà soát toàn bộ lịch sử mã nguồn (như trên GitHub), các nhánh, và công cụ CI/CD để phát hiện và ngăn chặn sự rò rỉ của các thông tin nhạy cảm này trước khi chúng bị khai thác. Giải pháp này thường được tích hợp vào quy trình phát triển, giúp phát hiện sớm các "bí mật" bị lộ, thông báo cho đội ngũ bảo mật và đưa ra các hành động khắc phục kịp thời (như thu hồi key) nhằm bảo vệ hạ tầng [29] [31].

#### **1.5.7. Dependency scanning**

Dependency scanning (quét phụ thuộc) là một quy trình bảo mật tự động thuộc giai đoạn phát triển phần mềm (thường tích hợp trong CI/CD), có chức năng kiểm tra các thư viện, package hoặc module nguồn mở mà ứng dụng sử dụng để phát hiện kịp thời các lỗ hổng bảo mật đã biết, phiên bản lỗi thời hoặc các rủi ro về giấy phép (license).

Công cụ này hoạt động bằng cách phân tích tệp cấu hình phụ thuộc (như package.json, pom.xml, requirements.txt) và so sánh chúng với các cơ sở dữ liệu lỗ hổng công khai (như CVE) để đưa ra báo cáo chi tiết, từ đó giúp nhóm phát triển (DevOps) ngăn chặn các nguy cơ an ninh ngay từ đầu, đảm bảo tính ổn định và tuân thủ các quy định bảo mật của ứng dụng [32] [33].

### 1.5.8. IaC và IaC scanning

Hạ tầng dạng mã nguồn (IaC) là phương pháp quản lý, cung cấp và vận hành hạ tầng IT (máy chủ, mạng, cơ sở dữ liệu) thông qua các tệp định nghĩa mã nguồn (code) thay vì cấu hình thủ công từng thiết bị. Bằng cách sử dụng các công cụ như Terraform, Ansible, hay AWS CloudFormation, IaC tự động hóa quy trình triển khai, đảm bảo tính nhất quán (consistency), giảm thiểu sai sót con người và cho phép kiểm soát phiên bản hạ tầng giống như mã phần mềm. Nhờ đó, các tổ chức có thể xây dựng, cập nhật và quản lý hạ tầng đám mây nhanh chóng, tin cậy và có khả năng mở rộng tốt hơn [34] [35].

IaC scanning là quy trình tự động kiểm tra, phân tích các tệp mã nguồn IaC để phát hiện cấu hình sai, lỗ hổng bảo mật hoặc các vấn đề tuân thủ trước khi mã được triển khai thực tế vào môi trường vận hành. Các công cụ quét phổ biến như Checkov, tfsec giúp đội ngũ DevOps tìm ra các rủi ro như mở cổng bảo mật quá mức, lưu trữ dữ liệu không mã hóa, hoặc lộ lọt thông tin nhạy cảm. Việc tích hợp IaC scanning vào quy trình CI/CD giúp bảo mật ngay từ bước phát triển (Shift Left Security), tiết kiệm chi phí sửa chữa và ngăn chặn sự cố bảo mật trong môi trường cloud [36] [37] [38].

### 1.5.9. Container image và SBOM:

Container Image là một gói phần mềm nhẹ, độc lập và có thể chạy được, bao gồm mọi thứ cần thiết (mã nguồn, thư viện, runtime<sup>6</sup>, cấu hình) để ứng dụng hoạt động nhất quán trên mọi môi trường. Được xây dựng dưới dạng các lớp read-only<sup>7</sup>, image

---

<sup>6</sup> Runtime: môi trường để phần mềm chạy

<sup>7</sup> Read-only: chỉ cho phép đọc và xem

đảm bảo tính di động cao, giúp các nhà phát triển đóng gói ứng dụng một lần và triển khai ở bất cứ đâu, từ máy cục bộ đến cloud [39] [40].

Container Scanning là quy trình kiểm tra tự động các image nhằm phát hiện lỗ hổng bảo mật (CVEs), cấu hình sai hoặc mã độc trước khi đưa vào môi trường thực tế. Quy trình này thường được tích hợp vào CI/CD pipeline, giúp quét từ các lớp base image<sup>8</sup> đến các thư viện ứng dụng, đảm bảo chỉ những image an toàn mới được đưa vào hoạt động [41] [42].

Signing Image<sup>9</sup> và SBOM là hai thành phần cốt lõi của bảo mật chuỗi cung ứng phần mềm. Signing sử dụng các công cụ như Cosign để tạo ra chữ ký số, xác thực tính toàn vẹn và nguồn gốc của image, đảm bảo không bị chỉnh sửa sau khi build [43]. SBOM đóng vai trò như một "danh sách thành phần" chi tiết, liệt kê mọi thư viện, dependency, cung cấp sự minh bạch để theo dõi lỗ hổng tiềm ẩn trong các thành phần phần mềm [44] [45].

Private Image Repository<sup>10</sup> (như Docker Registry, Harbor, AWS ECR) là kho lưu trữ riêng tư, an toàn, được dùng để quản lý, phân quyền truy cập và lưu trữ các container image nội bộ. Nó giúp doanh nghiệp bảo mật mã nguồn, tích hợp các hệ thống xác thực (SSO, LDAP), đồng thời đảm bảo quy trình CI/CD sử dụng các phiên bản image đã được kiểm duyệt và ký xác thực, tránh rò rỉ dữ liệu hoặc sử dụng image công cộng không an toàn [46] [47] [48].

#### 1.5.10. DAST

DAST<sup>11</sup> là phương pháp kiểm thử bảo mật "hộp đen" (black-box) được thực hiện bằng cách mô phỏng các cuộc tấn công thực tế (như SQL Injection, XSS) vào ứng dụng khi đang hoạt động nhằm phát hiện lỗ hổng bảo mật mà không cần truy cập vào mã nguồn. Các công cụ DAST gửi request HTTP đến ứng dụng và phân tích response để

---

<sup>8</sup> Base image: Hệ điều hành cơ sở

<sup>9</sup> Signing Image: Ký xác thực image

<sup>10</sup> Private Image Repository: Kho lưu trữ image riêng tư

<sup>11</sup> DAST: Kiểm thử bảo mật ứng dụng động

xác định các rủi ro bảo mật thực tế, đặc biệt hiệu quả trong việc tìm lỗi cấu hình, xác thực, hoặc các vấn đề liên quan đến hạ tầng, giúp ngăn chặn rò rỉ dữ liệu và tấn công từ bên ngoài [49] [50] [51].

## Chương II. THIẾT KẾ KIẾN TRÚC

### 1. Mô hình hóa các mối đe dọa

Trong quá trình xây dựng hệ thống DevSecOps, mô hình hóa các mối đe dọa là bước quan trọng giúp dự đoán và ngăn chặn sớm các rủi ro bảo mật. Thay vì chỉ kiểm tra ứng dụng ở bước cuối cùng, quy trình này rà soát toàn bộ vòng đời phát triển của phần mềm. Cụ thể, trong đồ án này, quy trình CI/CD được thiết kế để tự động xử lý 5 kịch bản rủi ro chính, tương ứng với các "chốt chặn".

Kịch bản đầu tiên (T1) là rủi ro lộ lọt thông tin nhạy cảm. Khi lập trình, nhà phát triển có thể vô tình gõ trực tiếp các mật khẩu, khóa API hoặc mã xác thực vào mã nguồn. Nếu đoạn mã này bị đẩy lên kho lưu trữ chung, tin tặc có thể lấy được và xâm nhập hệ thống. Để ngăn chặn, dự án sử dụng công cụ kiểm tra ngay trên máy tính của lập trình viên. Nếu phát hiện có chứa mật khẩu, hệ thống sẽ lập tức báo lỗi và chặn không cho đẩy mã nguồn đó lên mạng [52] [53].

Kịch bản thứ hai (T2) liên quan đến việc sử dụng các thư viện bên thứ ba bị lỗi. Việc dùng các thư viện mã nguồn mở có thể vô tình mang theo các lỗ hổng bảo mật nghiêm trọng (như các lỗi CVE) vào dự án. Để xử lý, hệ thống sẽ tự động quét các thành phần phụ thuộc này. Nếu phát hiện lỗ hổng nguy hiểm vượt mức cho phép, hệ thống sẽ chặn các yêu cầu sáp nhập mã (block PR merge), buộc lập trình viên phải cập nhật thư viện lên phiên bản an toàn thì mới được đi tiếp.

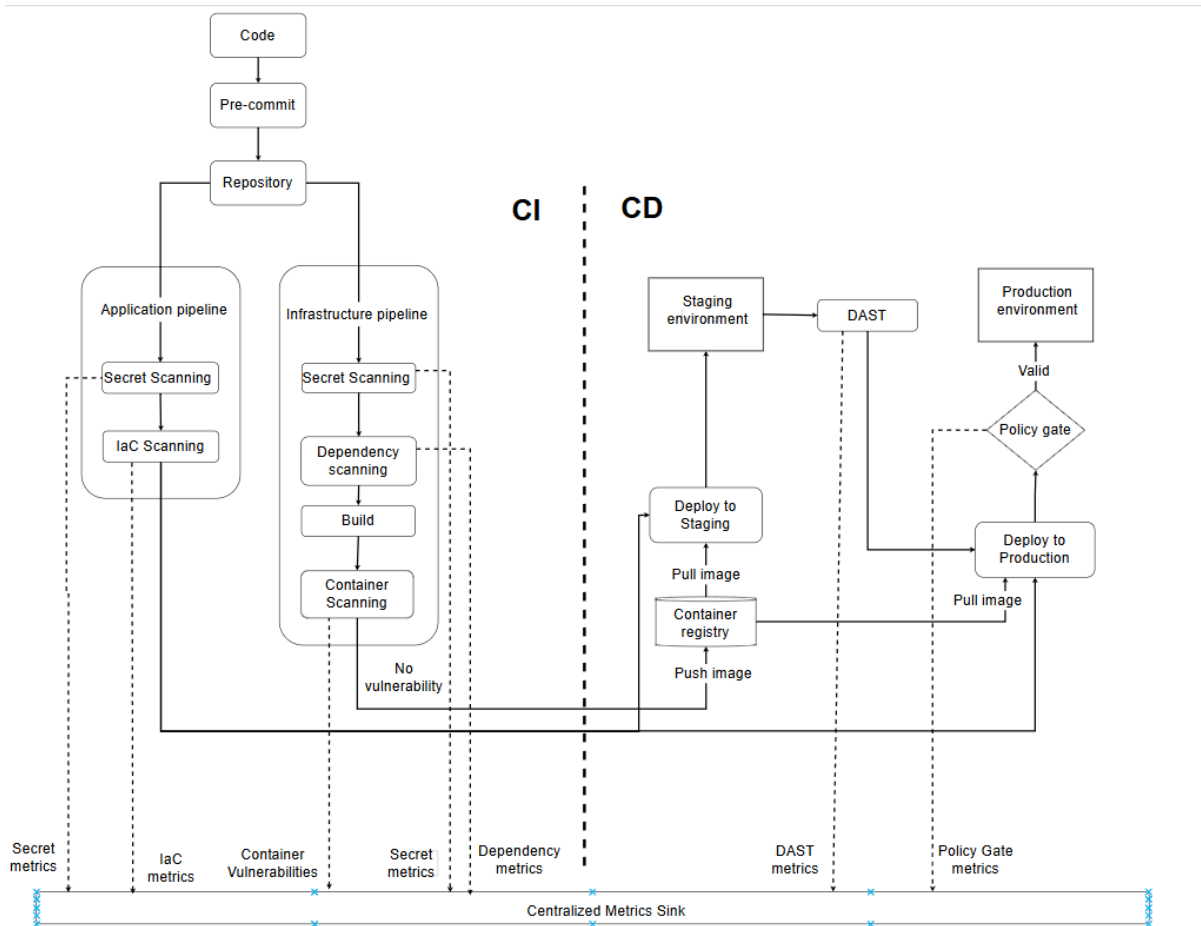
Kịch bản thứ ba (T3) tập trung vào các lỗi cấu hình hạ tầng đám mây (IaC). Những sơ suất nhỏ như lỡ tay mở quyền truy cập dữ liệu công khai hay mở sai cổng mạng có thể gây rò rỉ dữ liệu diện rộng. Hệ thống giải quyết bằng cách dùng công cụ tự động quét các file cấu hình. Khi phát hiện sai sót, hệ thống sẽ chặn việc sáp nhập mã. Thậm chí, với các lỗi thông thường, hệ thống có thể tự động tạo ra một bản sửa lỗi (auto-fix PR) để giúp lập trình viên khắc phục nhanh chóng.

Kịch bản thứ tư (T4) xử lý rủi ro từ các gói phần mềm (container image) kém an toàn. Nếu ứng dụng được đóng gói trên một hệ điều hành nền tảng đã cũ và chưa được vá lỗi, nó sẽ mang theo nhiều lỗ hổng ra môi trường chạy thực tế. Để kiểm soát, hệ

thống bắt buộc quét kiểm tra container image trước khi phân phối. Nếu image không đạt chuẩn an toàn, hệ thống sẽ cấm không cho lưu trữ image đó lên kho chính thức.

Cuối cùng, kịch bản thứ năm (T5) đối phó với các lỗ hổng chỉ xuất hiện khi ứng dụng đang chạy (runtime). Dù đã qua các bước kiểm tra mã tĩnh ở trên, một số lỗi nguy hiểm như cho phép tin tặc chạy mã độc từ xa (RCE) chỉ bộc lộ khi ứng dụng hoạt động trên môi trường chạy thử (staging). Lúc này, hệ thống sẽ kích hoạt công cụ kiểm thử động (DAST) để dò quét bề mặt ứng dụng. Nếu tìm thấy lỗ hổng nghiêm trọng, quy trình sẽ bị hủy bỏ ngay lập tức, ngăn không cho ứng dụng lỗi được đưa lên môi trường thật (Production).

## 2. Kiến trúc tổng quan



Hình 2: Kiến trúc tổng quan

Sơ đồ (Hình 2) mô tả một quy trình CI/CD được tích hợp chặt chẽ các bước kiểm tra bảo mật ở từng giai đoạn, hay còn gọi là kiến trúc DevSecOps. Luồng hoạt động

được chia thành hai giai đoạn phần chính là *CI* và *CD*, được hỗ trợ bởi một hệ thống thu thập dữ liệu tập trung nhằm giám sát và đánh giá toàn diện quá trình phát triển.

### 2.1. Giai đoạn đánh giá sơ bộ

Quy trình bắt đầu từ giai đoạn phát triển khi lập trình viên tiến hành viết. Trước khi mã nguồn được đẩy lên hệ thống, nó phải đi qua bước kiểm tra sơ bộ (Pre-commit) nhằm ngăn chặn các lỗi cơ bản hoặc vi phạm quy tắc định dạng, và cung cấp cho lập trình viên phản hồi nhanh. Sau khi vượt qua bước này, mã nguồn sẽ được đẩy lên và lưu trữ ở Repository, làm tiền đề để kích hoạt các luồng kiểm tra và tự động hóa tiếp theo trong giai đoạn CI. Việc phản hồi nhanh giúp lập trình viên nhận phản hồi nhanh trước khi push code lên repository, từ đó giảm đáng kể số lượng lỗi bảo mật đi vào pipeline chính.

### 2.2. Giai đoạn CI

Từ kho lưu trữ, luồng xử lý được chia thành hai luồng song song để kiểm tra toàn diện:

*Infrastructure pipeline*: Mã nguồn ứng dụng sẽ đi qua bước *Secret Scanning* để quét các thông tin nhạy cảm và quét bảo mật cấu hình hạ tầng qua bước *IaC Scanning*.

*Application pipeline*: Tương tự như luồng ứng dụng, nhánh này cũng thực hiện quét thông tin nhạy cảm - *Secret Scanning*. Sau đó, hệ thống tiếp tục quét các lỗ hổng từ các thư viện phụ thuộc trong bước *Dependency scanning*. Khi đảm bảo an toàn, mã sẽ được tiến hành đóng gói (bước *Build*) thành các Container và quét lỗ hổng trong Container (bước *Container Scanning*), nếu không tồn tại lỗ hổng, image sẽ được ký xác thực.

Chỉ khi hệ thống xác nhận mã nguồn và bộ chứa hoàn toàn - “No vulnerability”, phiên bản này mới được phép đẩy lên kho lưu trữ (*Container Registry*) để lưu trữ các phiên bản ứng dụng và chuẩn bị cho giai đoạn triển khai.

### 2.3. Giai đoạn CD

Khi image đã sẵn sàng trong kho lưu trữ, quy trình CD sẽ được kích hoạt với hai môi trường tách biệt:

*Môi trường Staging*: Hệ thống kéo image về môi trường Staging và triển khai. Tại đây, ứng dụng sẽ được kiểm thử bảo mật động (bước *DAST*). *DAST* giúp phát hiện các lỗ hổng trong lúc ứng dụng đang thực thi.

*Môi trường Production*: Sau khi hoàn thành kiểm thử tại Staging, image tiếp tục được kéo về và chuẩn bị triển khai lên Production. Tuy nhiên, trước khi chính thức áp dụng vào môi trường Production, image bắt buộc phải đi qua một Cổng chính sách (*Policy gate*). Chỉ khi các chính sách bảo mật và vận hành được đánh giá là hợp lệ - “Valid”, ứng dụng mới được phép triển khai lên môi trường Production.

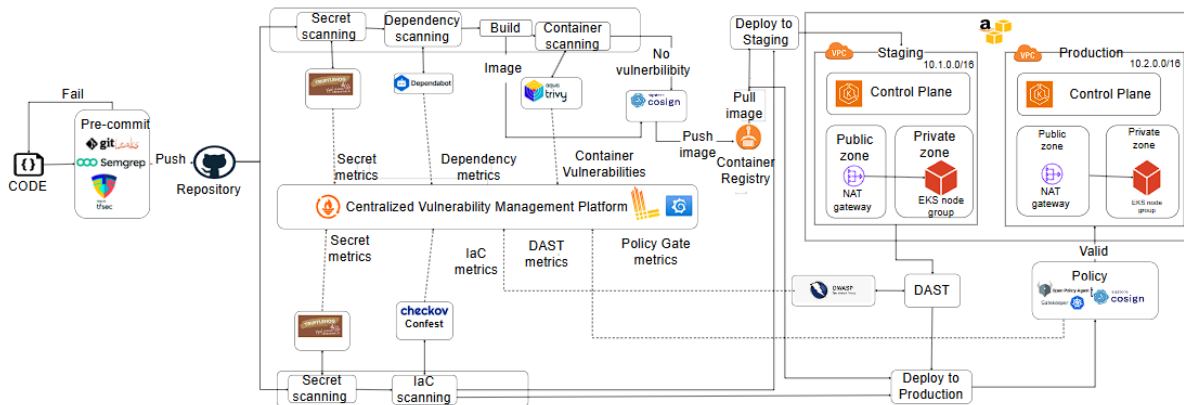
#### **2.4. Hệ thống lưu trữ tập trung:**

Xuyên suốt toàn bộ quy trình CI/CD, một hệ thống thu thập số liệu tập trung luôn hoạt động ngầm (được thể hiện bằng các đường nét đứt). Hệ thống này làm nhiệm vụ lắng nghe và lưu trữ các báo cáo từ mọi công cụ kiểm tra bảo mật, bao gồm:

- Số liệu về thông tin nhạy cảm - *Secret metrics*, từ cả hai luồng CI.
- Số liệu về lỗi cấu hình hạ tầng - *IaC metrics*.
- Các lỗ hổng trong bộ chứa - *Container Vulnerabilities* và các lỗ hổng các thư viện phụ thuộc - *Dependency metrics*.
- Kết quả từ hệ thống kiểm thử động - *DAST metrics*.
- Kết quả đánh giá từ Cổng chính sách - *Policy Gate metrics*.

Dữ liệu tổng hợp này giúp đội ngũ phát triển và quản trị bảo mật có cái nhìn toàn cảnh, dễ dàng truy vết sự cố và liên tục cải thiện chất lượng mã nguồn cũng như quy trình vận hành.

## Chương III: Triển khai



Hình 3: Kiến trúc triển khai

### 1. Công nghệ sử dụng

#### 1.1. GitHub và GitHub Actions

Trong giai đoạn triển khai hệ thống quản lý mã nguồn và tự động hóa quy trình phát triển phần mềm, nhóm đã tiến hành cấu hình và sử dụng nền tảng GitHub nhằm phục vụ quản lý source code tập trung, kiểm soát phiên bản và phối hợp phát triển giữa các thành viên. Toàn bộ mã nguồn của dự án đã được tổ chức theo cấu trúc repository chuẩn, bao gồm các nhánh phát triển chính như main, dev và các nhánh feature phục vụ phát triển chức năng riêng biệt. Việc áp dụng workflow Git giúp đảm bảo tính đồng bộ, dễ dàng rollback khi phát sinh lỗi và hỗ trợ kiểm soát lịch sử thay đổi mã nguồn.

Bên cạnh đó, hệ thống GitHub Actions đã được triển khai nhằm tự động hóa quy trình CI/CD cho dự án. Các workflow được cấu hình dưới dạng file YAML trong thư mục `.github/workflows`, cho phép tự động kích hoạt khi xảy ra các sự kiện như push code, pull request hoặc release phiên bản mới.

*Quản lý biến môi trường và secret key:* các biến môi trường được quản lý thông qua GitHub Secrets nhằm đảm bảo tính bảo mật cho thông tin xác thực và token triển khai. Cơ chế phân quyền repository đã được áp dụng để giới hạn quyền truy cập và thao tác đối với từng thành viên tham gia dự án.

*Triển khai ứng dụng lên môi trường staging và production:* Sau khi workflow build và test thành công, hệ thống có thể tự động thực hiện deploy lên server hoặc nền tảng cloud theo cấu hình định sẵn. Việc tự động hóa này giúp giảm đáng kể thời gian triển khai thủ công, hạn chế sai sót vận hành và nâng cao tính ổn định của quy trình phát hành.

## 1.2. Gitleaks

Nhằm tăng cường bảo mật mã nguồn và giảm thiểu nguy cơ rò rỉ thông tin nhạy cảm trong quá trình phát triển phần mềm, nhóm kỹ thuật đã tiến hành nghiên cứu và triển khai công cụ Gitleaks vào pre-commit và Secret scanning. Công cụ này được sử dụng để phát hiện các thông tin bí mật như API Key, Token, Password, SSH Key hoặc thông tin cấu hình nhạy cảm xuất hiện trong repository Git.

## 1.3. Semgrep

Semgrep là một công cụ phân tích mã nguồn tĩnh (SAST – Static Application Security Testing) được thiết kế để phát hiện lỗ hổng bảo mật, lỗi lập trình và các vấn đề tuân thủ trong mã nguồn ở nhiều ngôn ngữ lập trình khác nhau. Khác với các công cụ SAST truyền thống thường dựa trên phân tích cú pháp phức tạp hoặc xây dựng mô hình chương trình đầy đủ, Semgrep tập trung vào cơ chế “pattern matching” có cấu trúc, giúp việc viết quy tắc kiểm tra trở nên đơn giản nhưng vẫn đủ mạnh để phát hiện nhiều lớp vấn đề thực tế trong codebase hiện đại.

## 1.4. TruffleHog

TruffleHog là công cụ mã nguồn mở dùng để phát hiện các thông tin nhạy cảm (secrets) như API key, mật khẩu, token hoặc khóa truy cập bị lộ trong mã nguồn và lịch sử Git. Công cụ này không chỉ quét trạng thái hiện tại của repository mà còn phân tích toàn bộ lịch sử commit để tìm các chuỗi có khả năng chứa dữ liệu bí mật, thường dựa trên regex và phân tích entropy để xác định độ “giống secret”.

TruffleHog được tích hợp vào quy trình CI/CD trong đồ án nhằm phát hiện sớm rủi ro rò rỉ dữ liệu trước khi triển khai hệ thống. Nó có thể hoạt động với nhiều nền tảng như GitHub, nhưng cũng có nhược điểm là đôi khi tạo cảnh báo sai (false positive) và tiêu tốn tài nguyên khi quét các repository lớn nếu không được tối ưu cấu hình.

### 1.5. tfsec

tfsec là một công cụ phân tích tĩnh dành cho Terraform, được sử dụng để phát hiện các cấu hình hạ tầng có nguy cơ mất an toàn trước khi triển khai. Công cụ này quét các file Infrastructure as Code (IaC) và so sánh chúng với các bộ quy tắc bảo mật (security rules) dựa trên các tiêu chuẩn và best practices phổ biến như CIS benchmarks. Nhờ đó, tfsec giúp phát hiện sớm các lỗi cấu hình thường gặp như tài nguyên public không cần thiết, quyền IAM quá rộng, thiếu mã hóa dữ liệu hoặc cấu hình network không an toàn.

Trong trường hợp này, tfsec thường được tích hợp vào để tự động kiểm tra bảo mật mỗi khi có thay đổi mã nguồn. Kết quả quét có thể xuất ra nhiều định dạng như JSON hoặc SARIF để tích hợp với các nền tảng quản lý bảo mật. Ưu điểm chính của tfsec là tốc độ nhanh, dễ sử dụng và là công cụ ứng dụng của tư duy Shift-Left. Tuy nhiên, công cụ này chỉ phân tích ở mức cấu hình nên không thay thế được các giải pháp giám sát runtime hoặc kiểm thử bảo mật chuyên sâu.

### 1.6. Dependabot

Dependabot là một công cụ tự động hóa do GitHub phát triển nhằm hỗ trợ quá trình quản lý các thư viện, thành phần phụ thuộc trong dự án. Trong vòng đời DevSecOps, việc bảo mật các thư viện bên thứ ba là cực kỳ quan trọng để tránh lỗ hổng chuỗi cung ứng phần mềm. Công cụ này giúp tự động hóa quá trình giám sát, phát hiện và cập nhật các thành phần này một cách an toàn [54] [55].

Về cách thức hoạt động: Dependabot liên tục quét biểu đồ phụ thuộc của dự án và đối chiếu với cơ sở dữ liệu lỗ hổng bảo mật toàn cầu (GitHub Advisory Database). Khi phát hiện một thư viện đang sử dụng chứa lỗ hổng bảo mật hoặc có phiên bản cập nhật mới, Dependabot sẽ tự động tạo một Pull Request. Trong Pull Request này, nó sẽ cung cấp các thông tin chi tiết về mức độ rủi ro, mã CVE liên quan và đề xuất phiên bản an toàn nhất để khắc phục [54] [56].

**Ưu Điểm Nổi Bật:**

*Tự động hóa hoàn toàn:* Loại bỏ thao tác thủ công tìm kiếm các lỗ hổng CVE và tạo nhánh khắc phục.

*Cảnh báo sớm:* Ngăn chặn các lỗ hổng đã biết xâm nhập vào mã nguồn của ứng dụng.

*Tiết kiệm chi phí:* Là tính năng tích hợp sẵn của GitHub và hoàn toàn miễn phí cho cả kho lưu trữ công khai (public) và riêng tư (private).

### 1.6.1. Terraform

Terraform là công cụ cho phép các nhóm kỹ thuật định nghĩa và cung cấp hạ tầng trung tâm thông qua các tệp mã nguồn khai báo. Thay vì cấu hình thủ công dễ gây sai sót, Terraform tự động hóa việc khởi tạo mạng, máy chủ và cơ sở dữ liệu trên nhiều nền tảng đám mây. Nó được sử dụng trong DevSecOps vì có thể đảm bảo tính nhất quán của môi trường và cho phép triển khai hạ tầng lặp lại nhanh chóng, an toàn.

Việc tự động hóa hạ tầng đi kèm với các rủi ro bảo mật đặc thù nếu không được kiểm soát chặt chẽ. Một trong những sai lầm nghiêm trọng nhất là vô tình đẩy tệp trạng thái (State file) hoặc thông tin nhạy cảm (API keys, mật khẩu) lên Git. Ngoài ra, các cấu hình sai cơ bản (như mở cổng bảo mật quá mức, phân quyền IAM quá rộng) cũng có thể tạo ra các lỗ hổng hệ thống quy mô lớn.

### 1.7. Kubernetes

Kubernetes đóng vai trò là "nhạc trưởng" điều phối các ứng dụng container ở quy mô lớn. Trong môi trường DevSecOps, nó không chỉ là một nền tảng vận hành đơn thuần mà còn là lớp bảo vệ trung tâm. Tự động hóa việc mở rộng quy mô, khôi phục sự cố (failover) và cung cấp các mô hình triển khai liên mạch giúp các nhóm kỹ thuật tích hợp kiểm thử bảo mật vào mọi giai đoạn của vòng đời phần mềm (SDLC) một cách nhất quán.

### 1.8. Docker

Docker đóng vai trò cốt lõi trong mô hình DevSecOps bằng cách đóng gói ứng dụng và môi trường chạy vào các container. Nó không chỉ tự động hóa quy trình triển khai mà còn tích hợp các biện pháp bảo mật vào từng giai đoạn của vòng đời phát triển phần mềm (SDLC), chuyển đổi an ninh mạng từ một công đoạn riêng biệt thành quy trình xuyên suốt.

### 1.9. Checkov

Checkov là một công cụ phân tích tĩnh (Static Code Analysis) mã nguồn mở, hỗ trợ kỹ thuật Policy-as-Code (chính sách dưới dạng mã). Trong mô hình DevSecOps, Checkov đóng vai trò là lớp bảo mật đầu tiên, chuyên quét các tệp Infrastructure as Code (IaC) (như Terraform, CloudFormation, Kubernetes, Dockerfile) để phát hiện các lỗ hổng và cấu hình sai trước khi chúng được triển khai lên môi trường Cloud [57].

Checkov có thể được sử dụng trong đồ án vì tự động hóa nhờ hỗ trợ tích hợp linh hoạt với quy trình CI/CD như GitHub Actions. Công cụ được tích hợp vào IaC scanning, được thiết lập để chạy mỗi khi có yêu cầu hợp nhất code (Pull Request), tự động phân tích và đưa ra cảnh báo lỗi trực tiếp lên giao diện, giúp ngăn chặn ngay lập tức việc đưa cấu hình không an toàn vào các nhánh được thiết lập sẵn như prod (môi trường Production), staging (môi trường Staging) và các nhánh quan trọng như dev, main, master. Việc đưa Checkov vào giúp hiện thực hóa tư Shift-Left Security. Công cụ này cung cấp sẵn hàng trăm quy tắc bảo mật, giúp các nhóm phát hiện tự động các lỗ hổng như tài nguyên đám mây bị cấu hình công khai, thiếu mã hóa, hoặc lộ thông tin nhạy cảm. Điều này giúp giảm thiểu chi phí khắc phục sự cố nghiêm trọng và đảm bảo tuân thủ các tiêu chuẩn bảo mật ngay từ giai đoạn viết code.

### 1.10. Conftest

Conftest (viết tắt của Configuration Test) là một tiện ích kiểm thử cấu hình tĩnh cho dữ liệu có cấu trúc như YAML, JSON, hay INI. Được xây dựng dựa trên ngôn ngữ Rego của Open Policy Agent (OPA), công cụ này hỗ trợ định nghĩa chính sách bảo mật dưới dạng mã (Policy-as-Code), Conftest có khả năng đánh giá bảo mật đa nền tảng, bao gồm các tệp cấu hình của (Kubernetes), mã nguồn (Terraform), và định nghĩa (Dockerfile). Việc đánh giá diễn ra nhanh chóng, cho phép chặn quá trình build/deploy ngay khi hệ thống phát hiện các điểm bất thường dựa trên policy đã thiết lập.

Tuy nhiên, việc triển khai Conftest tạo ra tiêu chuẩn bảo mật đồng bộ, dễ dàng kiểm soát tập trung và tái sử dụng. Dù vậy, quy trình này cũng đối mặt với thách thức như độ phức tạp của ngôn ngữ Rego và nguy cơ gây ra số lượng lớn cảnh nếu không được tinh chỉnh hợp lý.

Conftest trong đồ án hoạt động như một chốt chặn tự động hóa, cho phép các đội ngũ phát triển và bảo mật kiểm tra các lỗ hổng ngay trong quy trình CI/CD trước khi triển khai. Công cụ giúp ngăn chặn rủi ro cấu hình sai có thể dẫn đến vi phạm dữ liệu.

### 1.11. Trivy

Trivy do Aqua Security phát triển là một công cụ quét bảo mật toàn diện và mã nguồn mở, được thiết kế chuyên biệt cho hệ sinh thái cloud-native. Trivy cho phép tự động hóa việc tìm kiếm các lỗ hổng bảo mật ngay từ giai đoạn đầu của vòng đời phát triển phần mềm (SDLC) [58].

Công cụ này được tích hợp trong đồ án vì khả năng quét đa mục tiêu, từ mã nguồn, container image, hệ thống tệp tin cho đến các cụm Kubernetes và cấu hình đám mây. Trivy không chỉ phát hiện các lỗ hổng hệ điều hành và phần mềm phụ thuộc (CVE) mà còn kiểm tra các lỗi cấu hình trong cơ sở hạ tầng dưới dạng mã (IaC), tìm kiếm các thông tin nhạy cảm bị lộ (Secrets như API keys, mật khẩu) và phân tích giấy phép phần mềm.

### 1.12. Cosign

Cosign là một công cụ mã nguồn mở đóng vai trò thiết yếu trong việc bảo mật chuỗi cung ứng phần mềm. Được phát triển bởi dự án Sigstore (thuộc tổ chức OpenSSF), Cosign cho phép các đội ngũ DevSecOps thực hiện ký và xác minh tính hợp lệ của các hiện vật phần mềm, đặc biệt là container images. Nó đảm bảo rằng phần mềm không bị can thiệp trái phép hoặc thay đổi trong toàn bộ vòng đời phát triển [59].

Cosign cung cấp hai phương thức ký và xác minh linh hoạt. Phương thức thứ nhất sử dụng các cặp khóa tĩnh truyền thống (Public/Private key) do người dùng tự quản lý. Phương thức thứ hai, nổi bật nhất của Sigstore là ký không cần khóa (keyless). Với keyless, Cosign sử dụng các mã thông báo OIDC (OpenID Connect) từ các nhà cung cấp như GitHub Actions, Google hay Microsoft để cấp các chứng chỉ ngắn hạn cho quá trình ký. Sau đó, dấu vết chữ ký và bằng chứng thời gian được lưu trữ minh bạch trên hệ thống Rekor [60] [61].

Việc tích hợp Cosign rất đơn giản, thường nằm ở bước cuối của tiến trình build và đẩy (push) image lên registry trong CI/CD. Các công cụ kiểm soát như OPA (Open

Policy Agent) có thể thiết lập chính sách chỉ cho phép triển khai các ứng dụng lên cluster khi chữ ký của Cosign được xác thực hợp lệ. Giải pháp này giúp đơn giản hóa việc triển khai hạ tầng zero-trust mà không cần quản lý các khóa mã hóa phức tạp [62].

### 1.13. Trục quan hóa dữ liệu - *Grafana*

*Grafana* đóng vai trò là giao diện hiển thị tập trung và là lớp tương tác tối cao của kiến trúc giám sát. Thay vì buộc đội ngũ quản trị an toàn thông tin và kỹ sư phát triển phần mềm phải truy cập vào từng công cụ riêng lẻ hoặc phân tích thủ công các tệp tin báo cáo định dạng dữ liệu thô như JSON hoặc HTML, *Grafana* kết nối trực tiếp đến các cơ sở dữ liệu lưu trữ phía dưới để tổng hợp và đồng bộ thông tin lên một bảng điều khiển duy nhất.

#### Các tính năng cốt lõi áp dụng trong đồ án:

*Hỗ trợ đa nguồn dữ liệu:* *Grafana* cho phép thực hiện các truy vấn đồng thời dữ liệu dạng chỉ số thời gian (Metrics) từ *Prometheus* và dữ liệu nhật ký sự kiện (Logs) từ *Loki* trên cùng một không gian làm việc.

*Trục quan hóa dữ liệu:* Hệ thống chuyển đổi các kết quả phát hiện lỗi hỏng thô thành các biểu đồ luồng dữ liệu, biểu đồ cột hoặc biểu đồ tròn, phân loại trục quan theo mức độ và biểu diễn xu hướng biến động của lỗi hỏng theo tiến trình thời gian.

*Cơ chế cảnh báo tự động:* Triển khai cấu hình các ngưỡng giới hạn an toàn. Khi số lượng lỗi hỏng mức độ nguy hiểm vượt quá định mức cho phép hoặc phát hiện thông tin nhạy cảm (Secrets) bị lộ lọt trong luồng CI, *Grafana* sẽ kích hoạt lệnh API gửi thông báo khẩn cấp về các kênh quản trị của nhóm phát triển.

### 1.14. Hệ thống thu thập chỉ số định lượng - *Prometheus*

*Prometheus* là một hệ thống mã nguồn mở chuyên dụng cho việc giám sát và lưu trữ dữ liệu dưới dạng chuỗi thời gian (Time-Series Metrics). Hệ thống hoạt động dựa trên mô hình dữ liệu đa chiều, trong đó các chỉ số được định danh bằng tên chỉ số (Metric name) kết hợp với các cặp khóa-giá trị (Labels). Cơ chế này cho phép hệ thống thực hiện các phép toán thống kê và truy vấn bộ dữ liệu lớn với độ trễ thấp thông qua ngôn ngữ chuyên dụng PromQL.

Trong kiến trúc DevSecOps, Prometheus được lựa chọn làm giải pháp cốt lõi để thu thập và quản lý các nhóm chỉ số định lượng, bao gồm chỉ số trạng thái thực thi của từng công đoạn quét bảo mật riêng biệt, mật độ lỗ hổng an ninh phân loại theo mức độ nghiêm trọng và tần suất phát hiện các sai sót cấu hình hạ tầng.

### **1.15. Hệ thống quản lý và phân tích nhật ký sự kiện - Grafana Loki**

Grafana Loki đóng vai trò là phân hệ lưu trữ và xử lý dữ liệu định tính dưới dạng logs. Loki được thiết kế dựa trên triết lý tối ưu hóa tài nguyên đám mây thông qua việc thiết lập cấu trúc chỉ mục tối giản. Thay vì tiến hành đánh chỉ mục cho toàn bộ nội dung văn bản thô của nhật ký, Loki chỉ thực hiện đánh chỉ mục cho các nhãn phân loại dữ liệu tương tự như cấu trúc thiết kế của Prometheus. Cơ chế độc lập này giúp giảm thiểu tối đa dung lượng lưu trữ hạ tầng, hạ thấp chi phí vận hành tài nguyên hệ thống và đẩy nhanh tốc độ thực thi các truy vấn tìm kiếm nhật ký trên diện rộng.

### **1.16. OWASP ZAP**

OWASP ZAP (Zed Attack Proxy) là công cụ kiểm thử bảo mật ứng dụng web mã nguồn mở hàng đầu, được thiết kế để phát hiện các lỗ hổng bảo mật trong quá trình ứng dụng đang chạy. Trong mô hình DevSecOps, công cụ này đóng vai trò là giải pháp kiểm thử bảo mật động (DAST), thực hiện các yêu cầu giả lập các cuộc tấn công vào ứng dụng đã được deploy, từ đó đánh giá mức độ an toàn thực tế của hệ thống. Quá trình quét thường được chia thành 3 mức độ: quét cơ bản (Baseline Scan), quét API, và quét chủ động (Full Scan) nhằm cân bằng giữa thời gian xử lý của pipeline và độ sâu của quá trình kiểm thử.

Để đảm bảo các lỗ hổng không bị lọt qua các môi trường tiếp theo, các ngưỡng bảo mật (security gates) được thiết lập trực tiếp trong file cấu hình pipeline. Quy trình CI/CD sẽ tự động đánh giá các báo cáo đầu ra, và nếu OWASP ZAP phát hiện các lỗ hổng ở mức độ nguy hiểm cao (High Risk) hoặc hơn, pipeline sẽ bị đình chỉ (fail build) để yêu cầu đội ngũ lập trình viên xử lý.

### **1.17. OPA Gatekeeper**

OPA Gatekeeper là công cụ thực thi chính sách bảo mật dạng mã (Policy-as-Code) được thiết kế riêng cho các môi trường Cloud-Native. Đóng vai trò là mảnh ghép

cốt lõi trong giai đoạn Security của quy trình DevSecOps, nền tảng này hoạt động như một "người gác cổng" tự động, giúp các tổ chức liên tục tuân thủ các tiêu chuẩn bảo mật và ngăn chặn các cấu hình sai sót trước khi tài nguyên được triển khai vào Kubernetes. Gatekeeper tích hợp sâu vào kiến trúc Kubernetes thông qua các Admission Controller. Khi có yêu cầu tạo hoặc cập nhật tài nguyên, hệ thống sẽ chặn các yêu cầu này và gửi thông tin đến Open Policy Agent (OPA) để đánh giá tính tuân thủ dựa trên ngôn ngữ lập trình Rego. Nếu phát hiện vi phạm, yêu cầu sẽ bị từ chối ngay lập tức, ngăn ngừa rủi ro xảy ra.

Điểm thách thức lớn nhất khi áp dụng OPA Gatekeeper là người vận hành phải làm quen với ngôn ngữ truy vấn Rego, đòi hỏi một quá trình học tập nhất định. Tuy nhiên, công cụ này cũng hỗ trợ hàng loạt các Constraint Template có sẵn, cho phép các nhóm DevOps nhanh chóng triển khai các tiêu chuẩn bảo mật trọng yếu mà không cần phải thành thạo viết mã ngay từ ngày đầu.

## **1.18. AWS**

Amazon Web Services (AWS) là nền tảng đám mây toàn diện và được sử dụng rộng rãi nhất trên thế giới, cung cấp hơn 200 dịch vụ đầy đủ tính năng từ các trung tâm dữ liệu trên toàn cầu. Được phát triển bởi Amazon.com, AWS là giải pháp dẫn đầu về thị phần, cung cấp hạ tầng vững chắc bao gồm tính toán, lưu trữ, cơ sở dữ liệu, phân tích, mạng, và bảo mật, phục vụ mọi đối tượng từ doanh nghiệp khởi nghiệp đến các tập đoàn lớn. Với mạng lưới hoạt động phủ khắp toàn cầu, AWS giúp doanh nghiệp triển khai ứng dụng, tối ưu hóa chi phí và tăng tốc độ sáng tạo một cách đáng kể [63] [64] [65].

### **1.18.1. AWS ECR**

AWS ECR đóng vai trò là một kho lưu trữ container tập trung, an toàn và được quản lý toàn diện. Đây là “trạm trung chuyển máu”, nơi các image sau khi được xây dựng và kiểm tra bảo mật sẽ được đẩy lên để chuẩn bị triển khai. ECR giúp các đội ngũ phát triển và vận hành tự động hóa việc lưu trữ, quản lý vòng đời và kiểm soát truy cập đối với các thành phần của ứng dụng.

ECR được sử dụng để lưu trữ image trong đồ án lần này vì bảo mật trong lưu trữ của ECR là yêu cầu bắt buộc, và ECR cung cấp khả năng phân quyền chi tiết thông qua AWS IAM. Các tổ chức có thể thiết lập chính sách (policies) xác định rõ ai hoặc dịch vụ nào được phép đẩy (push) hoặc kéo (pull) image. Đồng thời, mọi dữ liệu hình ảnh lưu trữ đều được mã hóa tự động ở trạng thái nghỉ (Encryption at Rest) bằng AWS KMS (sử dụng khóa do AWS quản lý hoặc khóa tự tạo), bảo vệ tài sản số khỏi các rò rỉ vật lý.

### 1.18.2. AWS EKS

Amazon EKS là dịch vụ Kubernetes được quản lý hoàn toàn trên AWS, giúp doanh nghiệp giảm tải việc vận hành các cụm máy chủ điều khiển (Control Plane). Trong hệ sinh thái DevSecOps, EKS đóng vai trò là nền tảng cốt lõi để tự động hóa, mở rộng quy mô và quản lý các ứng dụng dưới dạng microservices một cách linh hoạt, cho phép tích hợp dễ dàng các công cụ bảo mật vào vòng đời phát triển phần mềm [66].

EKS được ưu tiên để triển khai trong đồ án vì Xác thực và phân quyền (IAM) trên EKS được thắt chặt bằng cách áp dụng nguyên tắc đặc quyền tối thiểu. Các pod trên EKS không sử dụng IAM của node mà được gán quyền trực tiếp tới từng tài nguyên AWS thông qua IRSA (IAM Roles for Service Accounts) hoặc Pod Identity. Đồng thời, các chính sách như Kubernetes Network Policies hoặc Security Groups per Pod được áp dụng để phân tách luồng mạng và ngăn chặn sự di chuyển ngang (lateral movement) của các mối đe dọa.

## 2. Chi tiết triển khai

### 2.1. Workflow

Quy trình CI/CD được tổ chức theo kiến trúc GitHub Actions dạng multi-workflow modular, trong đó mỗi chức năng quan trọng của DevSecOps được tách thành một file workflow riêng trong thư mục *.github/workflows*. Cách tổ chức này giúp giảm độ phức tạp của từng pipeline đơn lẻ, đồng thời tăng khả năng tái sử dụng và bảo trì khi hệ thống mở rộng.



Hình 4: Các workflow trong Quy trình

Cấu trúc chính gồm các workflow chuyên biệt như *scan-app.yml*, *scan-iac.yml*, *build.yml*, *deploy.yml*, *provision.yml*, *policy\_gate.yml* và *zap.yml*. Mỗi workflow đảm nhiệm một vai trò rõ ràng trong vòng đời phần mềm, ví dụ scan bảo mật ứng dụng, kiểm tra hạ tầng IaC, build artifact, provisioning hạ tầng, triển khai ứng dụng, kiểm soát policy và kiểm thử bảo mật động. Việc tách nhỏ như vậy giúp mỗi pipeline có phạm vi trách nhiệm đơn nhất, dễ kiểm soát và dễ thay đổi độc lập.

Workflow trung tâm *main.yml* đóng vai trò là người điều phối. Thay vì chứa toàn bộ logic CI/CD, file này chỉ định nghĩa luồng điều phối bằng cách gọi lại các workflow con thông qua cơ chế:

```
Use: ./github/workflows/<tên workflow>.yaml
```

Bảng 1: Cơ chế gọi các workflow khác

Điều này biến *main.yml* thành một “controller pipeline”, chịu trách nhiệm sắp xếp thứ tự thực thi, định nghĩa dependency giữa các job và kiểm soát điều kiện chạy theo nhánh hoặc loại event.

Trong kiến trúc này, các workflow con được thiết kế theo dạng reusable workflows, cho phép truyền input và output giữa các bước. Ví dụ, workflow *build.yml* trả về *full\_image\_name*, sau đó được sử dụng trong *deploy.yml* để triển khai container chính xác phiên bản vừa build. Tương tự, *provision.yml* trả về thông tin cluster và namespace để các bước triển khai lên môi trường Staging hoặc Production có thể sử dụng lại mà không cần tính toán lại hạ tầng.

Cách tổ chức dependency giữa các workflow được quản lý thông qua *needs*, kết hợp với điều kiện *if* để kiểm soát luồng chạy theo trạng thái thành công hoặc thất bại của các bước trước đó. Điều này tạo ra một pipeline có tính ràng buộc chặt chẽ, đảm bảo rằng không có bước deploy nào được thực hiện nếu các bước security scan hoặc build chưa đạt yêu cầu.

Ngoài ra, hệ thống còn áp dụng cơ chế phân tách môi trường rõ ràng giữa staging và production ngay ở tầng workflow. Các workflow như *deploy.yml* và *provision.yml* nhận tham số *ENVIRONMENT* để thay đổi hành vi triển khai, giúp cùng một logic deploy nhưng có thể áp dụng linh hoạt cho nhiều môi trường khác nhau mà không cần viết lại pipeline.

Một điểm quan trọng trong cách tổ chức là việc tích hợp security workflows như *policy\_gate.yml* và *zap.yml* vào luồng CD. Thay vì tách rời hoàn toàn, các workflow này được chèn vào pipeline production như các “security checkpoints”, đảm bảo ứng dụng chỉ được phát hành sau khi đã vượt qua cả kiểm tra policy và kiểm thử DAST.

## 2.2. Hệ thống thu thập dữ liệu tập trung:

Hệ thống được xây dựng theo mô hình thu thập và quan sát tập trung trong đó Grafana Cloud đảm nhiệm vai trò nền tảng lưu trữ và trực quan hóa gồm hai thành phần chính là Prometheus (metrics) và Loki (logs).

Ở tầng metrics, Grafana Prometheus được sử dụng để tiếp nhận dữ liệu theo định dạng Influx Line Protocol thông qua HTTP endpoint `/api/v1/push/influx/write`. Mỗi loại scan sẽ sinh ra một metric riêng biệt, bao gồm số lượng lỗ hổng bảo mật từ Trivy (*vulnerabilities\_count*), số lượng secret phát hiện bởi TruffleHog (*secrets\_count*), số lỗi policy từ Conftest (*iac\_failures\_count*), và số cảnh báo từ OWASP ZAP (*dast\_alerts\_count*). Dữ liệu được trích xuất bằng `jq` từ các file JSON report và luôn có cơ chế fallback về 0 nếu file không tồn tại hoặc dữ liệu rỗng, đảm bảo pipeline không bị gián đoạn.

Song song với metrics, hệ thống còn triển khai lớp logging tập trung thông qua Grafana Loki. Các báo cáo scan chi tiết được chuyển thành JSON string và gửi lên Loki bằng API push dạng streams. Trước khi gửi, dữ liệu được tối ưu hóa để giảm dung

lượng, đặc biệt với Trivy report, chỉ giữ lại các trường quan trọng như Target, VulnerabilityID, Package Name, Installed Version và Severity. Điều này giúp tránh vượt giới hạn kích thước log của Loki (256KB) đồng thời vẫn đảm bảo đủ thông tin phục vụ phân tích.

Quá trình xử lý log được chuẩn hóa bằng cách sử dụng timestamp dạng nanosecond và chuyển đổi toàn bộ payload thành chuỗi an toàn JSON thông qua:

```
jq -Rs.
```

*Bảng 2: Lệnh chuyển đổi payload thành JSON*

Mỗi log stream đều được gắn metadata như repository, workflow, scan type và service name để hỗ trợ truy vấn sau này trong Grafana. Điều này giúp việc lọc log theo từng pipeline hoặc loại scan trở nên dễ dàng và có cấu trúc.

Hệ thống cũng được thiết kế theo hướng đa môi trường (multi-tenant), khi cùng lúc có thể đẩy dữ liệu lên Grafana Cloud chính và một instance Grafana phụ nếu các secrets tương ứng được cấu hình. Cơ chế kiểm tra điều kiện secret trước khi gửi giúp tránh lỗi runtime và đảm bảo pipeline vẫn hoạt động ngay cả khi một endpoint không khả dụng.

Về mặt độ tin cậy, toàn bộ các bước push metrics và logs đều được cấu hình với cơ chế “fail-safe”, nghĩa là lỗi khi gửi dữ liệu quan sát sẽ không làm fail toàn bộ pipeline CI/CD. Ngoài ra, các kiểm tra tồn tại file, kiểm tra giá trị null và fallback mặc định về 0 được sử dụng xuyên suốt nhằm đảm bảo tính ổn định của hệ thống thu thập dữ liệu.

### 2.3. Ứng dụng web mẫu

OWASP Juice Shop là một ứng dụng web mã nguồn mở được thiết kế với mục đích mô phỏng các lỗ hổng bảo mật phổ biến trong ứng dụng thực tế. Ứng dụng này thường được sử dụng trong đào tạo an toàn thông tin, kiểm thử xâm nhập (penetration testing) và thực hành các kỹ thuật khai thác lỗ hổng theo chuẩn OWASP. Juice Shop được xây dựng bằng Node.js cho backend và Angular cho frontend, tạo thành một ứng dụng single-page hiện đại, mô phỏng khá sát với các hệ thống thương mại điện tử thực tế.

Về mặt chức năng, Juice Shop hoạt động như một website bán hàng trực tuyến với đầy đủ các thành phần cơ bản như danh mục sản phẩm, giỏ hàng, thanh toán, tài khoản người dùng và hệ thống đánh giá. Tuy nhiên, điểm đặc biệt của ứng dụng này là nó cố tình chứa nhiều lỗi bảo mật ở nhiều mức độ khác nhau, từ cơ bản đến nâng cao. Người dùng có thể đăng ký tài khoản, đăng nhập, tìm kiếm sản phẩm và thực hiện các thao tác giống như một website thương mại điện tử thông thường, qua đó tạo môi trường tự nhiên để thực hành khai thác lỗ hổng.

Một đặc điểm quan trọng của Juice Shop là khả năng mô phỏng đa dạng các nhóm lỗ hổng theo chuẩn OWASP Foundation. Các lỗi này bao gồm SQL Injection, Cross-Site Scripting (XSS), Broken Authentication, Security Misconfiguration, và các vấn đề liên quan đến phân quyền truy cập. Mỗi lỗ hổng được thiết kế có chủ đích, đi kèm với các mức độ khó khác nhau, giúp người học có thể tiếp cận từ cơ bản đến nâng cao một cách có hệ thống.

Ngoài ra, Juice Shop còn có hệ thống “challenge” tích hợp sẵn, trong đó người dùng được giao nhiệm vụ tìm và khai thác các lỗ hổng cụ thể. Mỗi thử thách đều được ghi nhận điểm số, giúp theo dõi tiến độ học tập và khả năng khai thác lỗ hổng của người dùng. Cơ chế này biến ứng dụng thành một nền tảng học tập tương tác thay vì chỉ là một hệ thống demo tĩnh.

Tổng thể, OWASP Juice Shop đóng vai trò như một môi trường mô phỏng thực tế cao cho đồ án để nhóm hiểu rõ cách các lỗ hổng bảo mật xuất hiện trong ứng dụng web hiện đại. Đây là nền tảng phù hợp để đưa vào các đồ án liên quan đến an toàn thông tin, DevSecOps hoặc kiểm thử bảo mật ứng dụng, vì nó vừa có tính thực hành cao vừa phản ánh đúng các rủi ro phổ biến trong hệ thống thực tế.

## **2.4. IaC bằng Terraform/hashicorp**

Hạ tầng được triển khai bằng Terraform nhằm xây dựng một hệ thống AWS EKS phục vụ ứng dụng Juice Shop, với khả năng hỗ trợ nhiều môi trường gồm staging và production. Kiến trúc được thiết kế theo hướng phân tách môi trường rõ ràng, trong đó mỗi environment có một VPC riêng biệt, một dải CIDR riêng và bộ subnet riêng để đảm bảo cô lập hoàn toàn về mạng. Cụ thể, Staging sử dụng dải *10.1.0.0/16* trong khi

Production sử dụng *10.2.0.0/16*, giúp tránh xung đột IP và tạo điều kiện mở rộng hệ thống về sau.

Trong lớp mạng, hệ thống sử dụng module VPC chuẩn từ Terraform Registry để triển khai toàn bộ hạ tầng mạng trên AWS. Mỗi VPC được thiết kế với ba Availability Zones nhằm đảm bảo khả năng chịu lỗi đa vùng. Bên trong VPC, hệ thống được chia thành các public subnet và private subnet tương ứng cho từng môi trường. Public subnet được sử dụng cho các tài nguyên cần truy cập từ internet như load balancer, trong khi private subnet được dùng cho các node của Kubernetes cluster để tăng cường bảo mật. Ngoài ra, hệ thống còn bật NAT Gateway nhằm cho phép các tài nguyên trong private subnet có thể truy cập internet một chiều, phục vụ việc tải package hoặc cập nhật hệ thống.

Một điểm quan trọng trong thiết kế mạng là việc gắn Kubernetes subnet tagging, bao gồm các nhãn như `kubernetes.io/cluster` và `kubernetes.io/role`, giúp AWS Load Balancer Controller có thể tự động nhận diện và phân bổ tài nguyên đúng subnet khi tạo service kiểu LoadBalancer. Bên cạnh đó, VPC Flow Logs cũng được kích hoạt nhằm ghi lại toàn bộ lưu lượng mạng, hỗ trợ giám sát, phân tích và phục vụ mục đích audit bảo mật.

Ở lớp Kubernetes, hệ thống triển khai Amazon EKS phiên bản 1.31 sử dụng Terraform EKS module. Cluster được cấu hình với khả năng truy cập endpoint ở cả chế độ public và private, cho phép quản trị linh hoạt từ bên ngoài nhưng vẫn đảm bảo khả năng giao tiếp nội bộ trong VPC. EKS cluster cũng được cấu hình bật đầy đủ log control plane bao gồm API server, audit, authenticator, controller manager và scheduler, giúp tăng khả năng quan sát và debug hệ thống trong thực tế vận hành. Đồng thời, dữ liệu nhạy cảm trong cluster như secrets được mã hóa bằng AWS KMS key được tạo tự động trong quá trình triển khai.

Hệ thống Kubernetes được mở rộng bằng các add-ons tiêu chuẩn gồm VPC CNI, CoreDNS và kube-proxy. Các thành phần này đảm nhiệm vai trò mạng pod, phân giải DNS nội bộ và điều phối service networking trong cluster. Nhờ đó, cluster có thể hoạt động ổn định như một môi trường Kubernetes đầy đủ tính năng ngay sau khi khởi tạo mà không cần cấu hình thủ công thêm.

Phần compute của hệ thống sử dụng EKS Managed Node Group với instance type t3.small. Mô hình này giúp AWS tự động quản lý vòng đời node, bao gồm provisioning, scaling và replacement khi node gặp sự cố. Số lượng node được cấu hình linh hoạt theo từng môi trường, trong đó staging có 2 node và production có 3 node, với khả năng autoscaling từ 1 đến 3 node. Cách thiết kế này giúp tối ưu chi phí cho môi trường staging đồng thời đảm bảo hiệu năng cao hơn cho production.

Về bảo mật và truy cập, hệ thống sử dụng cơ chế authentication mode kết hợp API và ConfigMap nhằm tương thích cả phương pháp cũ và mới của EKS. Đồng thời, Terraform cấu hình access entries để tích hợp IAM principal dành cho GitHub Actions, cho phép pipeline CI/CD có thể truy cập và triển khai trực tiếp lên cluster. Quyền truy cập này được gán thông qua IAM policy, giúp kiểm soát quyền ở mức cluster scope.

Ngoài ra, hệ thống còn cấu hình provider AWS với default tags nhằm đảm bảo tất cả tài nguyên đều được gắn metadata như Environment, Project và Manager. Điều này giúp dễ dàng quản lý chi phí, truy vết tài nguyên và phân loại theo môi trường. Terraform cũng khai báo các provider hỗ trợ như Kubernetes, chuẩn bị cho việc triển khai ứng dụng lên cluster sau khi hạ tầng đã sẵn sàng.

## 2.5. Pre-commit

```
name: Pre-commit Security Gate

on:
  push:
  pull_request:
    branches:
      - main
      - dev
      - staging
      - prod

jobs:
  pre-commit-compliance:
    runs-on: ubuntu-latest

    steps:
      # Bước 1: Lấy mã nguồn mới nhất về Runner cloud
      - name: Checkout code
        uses: actions/checkout@v4
        with:
          fetch-depth: 0 # Cần thiết để Gitleaks có thể quét toàn bộ lịch sử
                           commit (commit history)
```

```
# Bước 2: Khởi tạo môi trường Python (vì framework pre-commit chạy trên nền Python)
- name: Set up Python
  uses: actions/setup-python@v5
  with:
    python-version: '3.11'

# Bước 3: Sử dụng Official Action để chạy pre-commit
- name: Run pre-commit hooks
  uses: pre-commit/action@v3.0.1
```

Bảng 3: Cấu hình workflow để dùng pre-commit

Cơ chế “Pre-commit Security Gate” được triển khai nhằm mục tiêu kiểm soát chất lượng và bảo mật mã nguồn ngay từ giai đoạn sớm trong vòng đời phát triển phần mềm. Hệ thống này được tích hợp trực tiếp vào GitHub Actions, đảm bảo mọi thay đổi mã nguồn khi được push hoặc tạo pull request vào các nhánh quan trọng như *main*, *dev*, *staging* và *prod* đều phải trải qua quá trình kiểm tra tự động trước khi được chấp nhận. Cách tiếp cận này giúp giảm thiểu rủi ro đưa mã lỗi hoặc chứa thông tin nhạy cảm vào môi trường chính thức.

Trong kiến trúc triển khai, GitHub Actions workflow đóng vai trò điều phối chính. Khi một sự kiện *push* hoặc *Pull Request* xảy ra, pipeline sẽ được kích hoạt trên môi trường *ubuntu-latest*<sup>12</sup>. Bước đầu tiên là checkout toàn bộ source code từ repository về runner. Việc sử dụng *fetch-depth: 0* cho phép tải toàn bộ lịch sử commit, điều này đặc biệt quan trọng đối với các công cụ như Gitleaks vì chúng cần phân tích cả lịch sử thay đổi để phát hiện các secret đã từng bị commit trước đó.

Sau khi mã nguồn được tải về, hệ thống thiết lập môi trường Python phiên bản 3.11 thông qua action *setup-python*. Đây là yêu cầu cần thiết vì framework pre-commit được xây dựng trên nền Python và các hook sẽ được thực thi trong môi trường này. Việc cố định phiên bản Python giúp đảm bảo tính ổn định và khả năng tái lập (reproducibility) của pipeline trong các lần chạy khác nhau.

Tiếp theo, GitHub Actions sử dụng *pre-commit/action* để kích hoạt cơ chế pre-commit hooks. Tại bước này, toàn bộ cấu hình trong file *.pre-commit-config.yaml* sẽ

<sup>12</sup>

Mượn máy ảo miễn phí từ GitHub để chạy workflow

được đọc và thực thi. Đây là thành phần cốt lõi của hệ thống, nơi định nghĩa tất cả các rule kiểm tra chất lượng và bảo mật được áp dụng lên mã nguồn.

```

repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
  - id: trailing-whitespace
  - id: end-of-file-fixer
  - id: check-yaml
  - id: check-added-large-files

- repo: https://github.com/gitleaks/gitleaks
  rev: v8.18.1
  hooks:
  - id: gitleaks

- repo: https://github.com/Yelp/detect-secrets
  rev: v1.4.0
  hooks:
  - id: detect-secrets
    args: [ '--baseline', '.secrets.baseline' ]
    exclude: package-lock.json

- repo: https://github.com/semgrep/semgrep
  rev: v1.65.0
  hooks:
  - id: semgrep
    args: [ '--config=auto', '--error' ]

- repo: https://github.com/aquasecurity/tfsec
  rev: v1.28.5
  hooks:
  - id: tfsec
    args: [ '--soft-fail' ]

```

Bảng 4: Cấu hình pre-commit trong file *.pre-commit-config.yaml*

Trong cấu hình pre-commit, nhóm hook đầu tiên đến từ repository *pre-commit-hooks*. Nhóm này tập trung vào các kiểm tra cơ bản về chất lượng mã nguồn như loại bỏ khoảng trắng thừa ở cuối dòng, đảm bảo file luôn kết thúc bằng newline, kiểm tra tính hợp lệ của file YAML và ngăn chặn việc commit các file có dung lượng quá lớn. Đây là lớp kiểm tra nền tảng giúp duy trì sự nhất quán trong codebase.

Bên cạnh đó, hệ thống tích hợp Gitleaks như một công cụ quét bảo mật quan trọng. Gitleaks có khả năng phân tích toàn bộ lịch sử commit để phát hiện các thông tin nhạy cảm như API key, token hoặc mật khẩu bị lộ. Điều này giúp ngăn chặn nguy cơ rò rỉ dữ liệu ngay cả khi secret đã từng bị commit trong quá khứ.

Song song với Gitleaks, detect-secrets của Yelp được sử dụng để phát hiện các secret tồn tại trong code hiện tại. Công cụ này hoạt động dựa trên cơ chế baseline, cho phép kiểm soát và loại trừ các trường hợp hợp lệ, đồng thời tập trung phát hiện các chuỗi có khả năng là credential bị hardcode trong source code. Việc kết hợp cả hai công cụ giúp tăng độ bao phủ trong việc phát hiện secrets ở nhiều mức độ khác nhau.

Hệ thống cũng tích hợp Semgrep như một công cụ phân tích tĩnh bảo mật ứng dụng (SAST). Semgrep sử dụng các rule-based engine để phát hiện các pattern code không an toàn như SQL injection, insecure function usage hoặc các lỗi logic có thể dẫn đến lỗ hổng bảo mật. Với cấu hình `--config=auto`, Semgrep tự động lựa chọn bộ rule phù hợp với ngôn ngữ và framework trong dự án, đồng thời chế độ `--error` đảm bảo pipeline sẽ fail nếu phát hiện vi phạm nghiêm trọng.

Ngoài ra, tfsec được đưa vào để kiểm tra bảo mật cho Infrastructure as Code, đặc biệt là Terraform. Công cụ này giúp phát hiện các cấu hình không an toàn như bucket S3 public, security group mở toàn bộ truy cập hoặc IAM policy quá rộng. Tuy nhiên, với tùy chọn `--soft-fail`, các lỗi phát hiện chỉ được coi là cảnh báo thay vì chặn pipeline, cho phép đội ngũ phát triển có thể cân bằng giữa bảo mật và tính linh hoạt trong triển khai hạ tầng.

Tổng thể, luồng hoạt động của hệ thống bắt đầu từ việc trigger sự kiện trên GitHub, sau đó lần lượt thực hiện checkout code, thiết lập môi trường Python, và kích hoạt pre-commit framework. Framework này sẽ đọc cấu hình và chạy tuần tự các nhóm kiểm tra từ format code, phát hiện secrets, phân tích bảo mật ứng dụng đến kiểm tra cấu hình hạ tầng. Nếu bất kỳ bước nào thất bại, pipeline sẽ bị dừng và ngăn chặn việc merge code vào các nhánh chính.

Mặc dù hệ thống hiện tại đã tương đối đầy đủ về mặt bảo mật và chất lượng code, vẫn tồn tại một điểm cần lưu ý về cấu trúc cấu hình. Phần định nghĩa *repos*: thuộc về pre-commit configuration và cần được tách riêng vào file `.pre-commit-config.yaml` thay vì nằm chung trong GitHub Actions workflow. Việc tách biệt này giúp đảm bảo đúng chuẩn của pre-commit framework và tránh lỗi khi pipeline được thực thi.

## 2.6. Secret scanning:

Mục tiêu chính của Secret scanning là phát hiện các thông tin nhạy cảm bị lộ trong mã nguồn ứng dụng, bao gồm API keys, access tokens, credentials, private keys và các chuỗi bí mật khác có thể gây rủi ro bảo mật nếu bị khai thác.

```
- name: Checkout Source Code
  uses: actions/checkout@v4
  with:
    fetch-depth: 0
- name: Run TruffleHog Scan (JSON Output)
  run: |
    docker run --rm -v "$PWD:/pwd" trufflesecurity/trufflehog:latest
    filesystem /pwd --json > trufflehog-report.json
```

Bảng 5: Cấu hình chính trong Secret scanning

Quy trình Secret Scan được thực hiện bằng công cụ TruffleHog, chạy trong môi trường container Docker để đảm bảo tính nhất quán và cô lập môi trường thực thi. Toàn bộ source code của repository được checkout với *fetch-depth: 0*, cho phép TruffleHog truy xuất toàn bộ lịch sử commit thay vì chỉ scan snapshot hiện tại. Điều này giúp tăng khả năng phát hiện các secret đã từng tồn tại trong lịch sử Git nhưng có thể đã bị xóa khỏi phiên bản hiện tại của mã nguồn.

Sau khi mã nguồn được tải về, TruffleHog thực hiện quét toàn bộ filesystem trong thư mục làm việc và xuất kết quả dưới dạng JSON thông qua file *trufflehog-report.json*. Việc sử dụng định dạng JSON giúp chuẩn hóa dữ liệu đầu ra, tạo điều kiện thuận lợi cho việc xử lý tự động trong các bước tiếp theo của pipeline như thống kê, giám sát và lưu trữ.

Kết quả của quá trình scan được lưu lại dưới dạng GitHub Actions artifact. Artifact này đóng vai trò như một bằng chứng kiểm toán (audit evidence), cho phép đội ngũ bảo mật hoặc phát triển có thể tải về và phân tích lại khi cần thiết. Điều này đặc biệt quan trọng trong các hệ thống yêu cầu tuân thủ (compliance) hoặc truy vết lịch sử bảo mật.

Sau bước lưu trữ, pipeline tiến hành xử lý dữ liệu bằng công cụ `jq` để phân tích file JSON và tính toán tổng số lượng secret được phát hiện. Mỗi object trong file kết quả tương ứng với một phát hiện bảo mật, từ đó tổng số secret được sử dụng như một chỉ số (metrics) phản ánh mức độ rủi ro của repository tại thời điểm scan.

Chỉ số này được chuyển đổi sang định dạng Influx Line Protocol và gửi tới hệ thống Prometheus/Grafana Cloud. Metric bao gồm các nhãn như repository, branch và loại scan (*secret-scan*), cùng với giá trị *secrets\_count*. Cách thiết kế này cho phép hệ thống giám sát theo dõi xu hướng phát sinh secret theo thời gian, đồng thời hỗ trợ cảnh báo khi số lượng secret vượt ngưỡng cho phép.

Bên cạnh metric, toàn bộ kết quả scan cũng được đẩy sang hệ thống log tập trung Loki. Pipeline thực hiện chuyển đổi JSON report thành chuỗi log an toàn, sau đó đóng gói vào định dạng stream của Loki kèm timestamp theo đơn vị nanosecond, đảm bảo khả năng truy vết chi tiết từng lần quét, bao gồm thông tin repository, branch và workflow liên quan, giúp phục vụ phân tích điều tra bảo mật khi cần thiết.

Một đặc điểm quan trọng của Secret Scanning trong pipeline là cơ chế “always-run”. Tất cả các bước xử lý metric và logging đều được cấu hình *if: always()* nhằm đảm bảo dữ liệu telemetry luôn được gửi đi bất kể kết quả scan thành công hay thất bại. Điều này giúp hệ thống giám sát có dữ liệu liên tục và không bị gián đoạn bởi lỗi trong quá trình CI/CD.

## 2.7. Dependency scanning

Việc triển khai Dependency scanning bằng Dependabot trong hệ thống nhằm mục tiêu tự động hóa việc phát hiện và xử lý các lỗ hổng bảo mật liên quan đến thư viện phụ thuộc. Giải pháp này được tích hợp trực tiếp trong nền tảng GitHub, giúp theo dõi các dependency trong dự án và đối chiếu với cơ sở dữ liệu lỗ hổng bảo mật đã biết (CVE). Khi phát hiện rủi ro, hệ thống sẽ đưa ra cảnh báo hoặc tự động tạo Pull Request để cập nhật phiên bản an toàn hơn.

Trong quá trình vận hành, Dependabot sẽ tự động phân tích các file khai báo dependency như *package.json*, *requirements.txt* hoặc *pom.xml* để xác định phiên bản thư viện đang sử dụng. Sau đó, hệ thống so sánh với dữ liệu lỗ hổng bảo mật và xác định mức độ ảnh hưởng. Các cảnh báo thường được phân loại theo mức độ nghiêm trọng như high và critical, giúp nhóm phát triển ưu tiên xử lý các vấn đề quan trọng trước.

Đối với các cảnh báo có mức độ ảnh hưởng thấp, đặc biệt là các dependency chỉ sử dụng trong môi trường phát triển hoặc kiểm thử, hệ thống có thể cấu hình để tự động bỏ qua hoặc giảm mức độ ưu tiên. Điều này giúp giảm nhiều cảnh báo và tập trung nguồn lực vào các rủi ro có khả năng ảnh hưởng đến môi trường production.

Khi phát hiện dependency cần cập nhật, Dependabot sẽ tự động tạo Pull Request chứa thay đổi phiên bản tương ứng. Pull Request này thường đi kèm thông tin về phiên bản mới, lý do cập nhật và các ghi chú liên quan đến thay đổi bảo mật. Quy trình này giúp lập trình viên dễ dàng review, kiểm thử và merge thay đổi theo quy trình CI/CD hiện có.

## 2.8. IaC scanning

Trong hệ thống IaC Security Scanning, bên cạnh các công cụ như Checkov và Conftest CLI trong CI pipeline, một lớp policy-as-code được xây dựng bằng Rego (OPA v1) đóng vai trò định nghĩa các quy tắc bảo mật tĩnh (static compliance rules). Các policy này được thiết kế để kiểm soát cả hai miền chính: Kubernetes workload security và Terraform infrastructure security, đảm bảo hạ tầng không chỉ đúng cú pháp mà còn tuân thủ các nguyên tắc bảo mật và chi phí vận hành.

Ở lớp Kubernetes policy, hệ thống áp dụng quy tắc kiểm soát việc expose dịch vụ ra Internet. Cụ thể, các Service có *type=LoadBalancer* sẽ bị đánh dấu vi phạm trong môi trường CI vì có nguy cơ tạo public endpoint trực tiếp. Điều này giúp ngăn chặn việc vô tình mở tài nguyên nội bộ ra Internet ngay từ giai đoạn quét tĩnh, trước khi triển khai thực tế. Đây là một lớp kiểm soát quan trọng trong mô hình zero-trust infrastructure.

Tiếp theo, policy Kubernetes còn áp dụng các yêu cầu hardening container. Một trong các rule bắt buộc là container phải bật *readOnlyRootFilesystem=true*, nhằm giảm thiểu khả năng ghi dữ liệu vào filesystem và hạn chế nguy cơ bị lợi dụng khi có exploit. Bên cạnh đó, container cũng phải thu hồi toàn bộ Linux capabilities bằng cách yêu cầu *capabilities.drop* chứa giá trị "ALL", nhằm loại bỏ các quyền hệ thống không cần thiết.

Ngoài ra, policy còn kiểm soát hành vi nâng quyền (privilege escalation). Cụ thể, container trong Deployment phải thiết lập *allowPrivilegeEscalation=false*, đảm bảo

tiến trình bên trong container không thể nâng quyền lên mức cao hơn, giảm thiểu rủi ro khi container bị xâm nhập. Các rule này kết hợp tạo thành một lớp container security baseline trong toàn bộ hệ thống Kubernetes.

Ở lớp Terraform, policy Conftest được sử dụng để kiểm soát cấu hình hạ tầng AWS trước khi triển khai. Một trong những rule quan trọng là cấm bật public endpoint cho EKS cluster thông qua thuộc tính `cluster_endpoint_public_access=true`. Điều này giúp đảm bảo control plane của Kubernetes không bị expose ra Internet, tuân thủ nguyên tắc bảo mật network segmentation.

Bên cạnh đó, hệ thống còn áp dụng rule kiểm soát chi phí và hiệu năng bằng cách giới hạn instance type trong node group. Cụ thể, các instance type bắt đầu bằng ký tự “p” (thường đại diện cho GPU/compute-intensive instances) sẽ bị đánh dấu vi phạm trong môi trường CI. Đây là cách để ngăn chặn việc vô tình provisioning các tài nguyên chi phí cao trong môi trường không phù hợp như staging hoặc test.

Một yêu cầu khác trong policy Terraform là bắt buộc cấu hình `default_tags` trong AWS provider. Rule này đảm bảo mọi tài nguyên AWS được gắn metadata chuẩn hóa như environment, owner hoặc project, từ đó hỗ trợ tốt hơn cho việc tracking, cost allocation và governance. Nếu provider block không chứa `default_tags`, pipeline sẽ coi đó là vi phạm policy.

Để xác định chính xác input Terraform, policy sử dụng cơ chế nhận diện dựa trên cấu trúc dữ liệu như `terraform`, `provider`, `module` hoặc `resource`. Đồng thời, các hàm helper như `walk()` và `path_ends_with()` giúp policy có khả năng kiểm tra sâu vào các nested attributes mà không phụ thuộc vào cấu trúc cố định.

## 2.9. Buid và Container scanning

Workflow GitHub Actions “Build and Container Security” được thiết kế như một pipeline CI/CD tích hợp các cơ chế DevSecOps nhằm đảm bảo quá trình xây dựng, kiểm thử và phát hành Docker image được kiểm soát chặt chẽ cả về chất lượng lẫn bảo mật. Các tham số đầu vào quan trọng như `AWS_REGION` được truyền vào để đảm bảo tính linh hoạt khi triển khai trên nhiều môi trường AWS khác nhau.

Trong giai đoạn khởi tạo, pipeline thiết lập các biến môi trường cốt lõi như phiên bản Node.js, tên repository ECR và tên ứng dụng. Sau đó, mã nguồn được checkout từ repository GitHub để chuẩn bị cho quá trình build. Node.js version 24 được cài đặt kèm cơ chế cache npm nhằm tối ưu thời gian cài đặt dependencies, giúp giảm đáng kể thời gian thực thi pipeline trong các lần chạy lặp lại.

Tiếp theo, pipeline tiến hành cấu hình quyền truy cập AWS thông qua access key và secret key được lưu trữ trong GitHub Secrets. Sau khi xác thực thành công, hệ thống thực hiện đăng nhập vào Amazon Elastic Container Registry (ECR) để chuẩn bị cho quá trình push Docker image. Đây là bước quan trọng đảm bảo pipeline có quyền tương tác với registry container nội bộ trên AWS.

Một bước quan trọng trong pipeline là chuẩn hóa tên image. Image được đặt theo format bao gồm registry, repository, tên ứng dụng và commit SHA của GitHub. Việc sử dụng Git commit hash làm version tag giúp đảm bảo tính bất biến của image, đồng thời tăng khả năng truy vết (traceability) trong toàn bộ vòng đời phần mềm. Thông tin này được lưu vào output để các bước tiếp theo có thể sử dụng nhất quán.

Sau đó, Docker Buildx được kích hoạt để hỗ trợ BuildKit, giúp tối ưu quá trình build image với cơ chế caching nâng cao. Image được build từ thư mục `app/` và được load vào local Docker daemon mà chưa push lên registry. Cơ chế cache từ GitHub Actions được sử dụng nhằm giảm thời gian build trong các lần chạy tiếp theo.

Khi image đã được build xong, pipeline tiến hành quét bảo mật bằng Trivy. Công cụ này thực hiện phân tích các lỗ hổng trong hệ điều hành và thư viện của image, tập trung vào mức độ nghiêm trọng HIGH và CRITICAL. Kết quả được xuất ra dưới nhiều định dạng khác nhau, bao gồm JSON để xử lý dữ liệu, và SARIF để tích hợp trực tiếp với GitHub Security Tab. Việc đặt *exit-code: 0* đảm bảo pipeline không bị gián đoạn ngay cả khi phát hiện lỗ hổng, nhằm duy trì luồng CI/CD liên tục và chuyển việc xử lý lỗi sang các hệ thống giám sát hoặc bước kiểm thử khác.

Song song với đó, Trivy cũng được sử dụng để tạo báo cáo phục vụ phân tích và lưu trữ. Các báo cáo này được upload dưới dạng artifact của GitHub Actions, giúp phục vụ việc audit hoặc kiểm tra lại khi cần thiết. Đồng thời, SARIF report được đẩy lên

GitHub Security Dashboard, cho phép hiển thị trực quan các lỗ hổng trong giao diện bảo mật của repository.

Sau bước quét bảo mật, image Docker được push lên Amazon ECR, đánh dấu việc hoàn tất quá trình xây dựng và kiểm tra. Trước khi push, image đã trải qua toàn bộ quá trình scan nhằm đảm bảo chất lượng và giảm thiểu rủi ro đưa image lỗi lên registry.

Pipeline tiếp tục thực hiện sinh Software Bill of Materials (SBOM) bằng công cụ Syft. SBOM này mô tả toàn bộ thành phần phần mềm bên trong container, bao gồm hệ điều hành, thư viện hệ thống và các dependency của ứng dụng. Kết quả được xuất ra dưới định dạng SPDX JSON và lưu lại dưới dạng artifact để phục vụ các yêu cầu tuân thủ và kiểm toán chuỗi cung ứng phần mềm.

Một phần quan trọng khác của pipeline là việc tích hợp quan sát bảo mật (security observability). Số lượng lỗ hổng được trích xuất từ báo cáo Trivy bằng công cụ jq và được chuyển thành metric. Metric này sau đó được đẩy lên Grafana Prometheus theo định dạng Influx Line Protocol, giúp xây dựng dashboard theo dõi xu hướng lỗ hổng theo thời gian. Đồng thời, pipeline cũng gửi log chi tiết của Trivy sang Grafana Loki, nhưng đã được tối ưu hóa để giảm kích thước payload nhằm tránh vượt giới hạn hệ thống logging.

## 2.10. Triển khai hạ tầng

Workflow “Provision” được thiết kế để tự động hóa việc triển khai hạ tầng AWS theo mô hình Infrastructure as Code với Terraform, đồng thời tích hợp trực tiếp vào GitHub Actions để đảm bảo quá trình provisioning có thể chạy thủ công hoặc được gọi từ các workflow khác. Workflow cho phép truyền vào các tham số quan trọng như *ENVIRONMENT* và *AWS\_REGION*, giúp tái sử dụng pipeline cho nhiều môi trường khác nhau như dev, staging hoặc production.

Trong phần cấu hình quyền (permissions), workflow được cấp các quyền cần thiết để thao tác với tài nguyên GitHub và AWS một cách an toàn trong CI/CD context. Cụ thể, *contents: write* cho phép cập nhật nội dung repository nếu cần, *id-token: write* phục vụ cho cơ chế xác thực OIDC trong các kịch bản mở rộng, *security-events: write* hỗ trợ tích hợp các công cụ security scanning, và *packages: read* cho phép đọc các

package từ GitHub registry. Đây là cấu hình phù hợp cho pipeline hạ tầng có tính chất tự động và yêu cầu quyền truy cập tài nguyên hệ thống.

Workflow sử dụng một tập hợp biến môi trường để chuẩn hóa cấu hình triển khai. Các biến này bao gồm thông tin về namespace Kubernetes mặc định, tên cluster cơ sở, môi trường triển khai, và region AWS. Bên cạnh đó là các biến quan trọng cho Terraform backend như S3 bucket, region của S3 và DynamoDB table dùng để locking state. Việc tách riêng các biến này giúp đảm bảo pipeline có thể hoạt động linh hoạt theo từng môi trường mà không cần thay đổi logic chính của workflow.

Job chính của pipeline là “Provision Infrastructure on AWS”, chạy trên môi trường *ubuntu-latest*. Job này chịu trách nhiệm thực thi toàn bộ quy trình từ chuẩn bị môi trường, cấu hình AWS, khởi tạo Terraform backend cho đến triển khai hạ tầng EKS. Đồng thời job cũng xuất ra hai output quan trọng là *CLUSTER\_NAME* và *K8S\_NAMESPACE*, phục vụ cho các workflow downstream như deploy ứng dụng hoặc cấu hình Kubernetes.

Workflow cấu hình quyền truy cập AWS bằng action *aws-actions/configure-aws-credentials@v4* sử dụng access key và secret key được lưu trong GitHub Secrets. Điều này cho phép runner có quyền tương tác với các dịch vụ AWS như S3, DynamoDB và EKS trong quá trình triển khai. Song song đó, AWS CLI v2 cũng được cài đặt thủ công để phục vụ các thao tác kiểm tra và tạo tài nguyên backend.

Một bước quan trọng là bootstrap Terraform backend, trong đó workflow kiểm tra sự tồn tại của S3 bucket dùng để lưu state file. Nếu bucket chưa tồn tại, hệ thống sẽ tự động tạo mới và bật versioning để đảm bảo khả năng khôi phục trạng thái. Đồng thời, DynamoDB table cũng được kiểm tra và tạo nếu chưa tồn tại, với cấu hình primary key là *LockID* nhằm hỗ trợ cơ chế state locking của Terraform, tránh xung đột khi có nhiều pipeline chạy đồng thời.

Sau khi backend được đảm bảo, workflow thực hiện bước dọn dẹp Terraform lock nếu tồn tại lock cũ từ các lần chạy trước bị lỗi hoặc gián đoạn. Việc này giúp tránh tình trạng state bị khóa vĩnh viễn, gây cản trở các lần deploy tiếp theo. Tiếp đó, pipeline thiết lập các biến output nội bộ như *CLUSTER\_NAME* và *K8S\_NAMESPACE*, trong đó

tên được ghép từ base name và môi trường triển khai nhằm đảm bảo tính tách biệt giữa các môi trường.

Workflow sau đó thiết lập các biến môi trường dành riêng cho Terraform, bao gồm region AWS, tên cluster, môi trường và các ARN IAM cần thiết. Những biến này được Terraform sử dụng để tạo và cấu hình tài nguyên AWS tương ứng. Đồng thời biến *TF\_IN\_AUTOMATION* được bật để tối ưu hóa hành vi Terraform trong môi trường CI/CD, hạn chế các prompt tương tác.

Bước Terraform init được thực hiện để khởi tạo backend remote trên S3 và cấu hình locking với DynamoDB. Tại bước này, Terraform sẽ đồng bộ state với backend trung tâm, đảm bảo mọi thay đổi hạ tầng được quản lý tập trung và nhất quán giữa các lần chạy pipeline.

Tiếp theo là bước Terraform plan cho hạ tầng EKS. Pipeline sử dụng chế độ *-detailed-exitcode* để phân biệt ba trạng thái: không có thay đổi, có thay đổi hoặc lỗi. Kết quả của bước này được lưu lại để quyết định có cần thực hiện apply hay không. Nếu phát hiện drift hoặc thay đổi hạ tầng, pipeline sẽ tiếp tục sang bước apply; nếu không có thay đổi, hệ thống vẫn ghi nhận trạng thái thành công và giữ nguyên hạ tầng hiện tại.

Cuối cùng, Terraform apply được thực thi để áp dụng các thay đổi vào AWS nếu cần thiết. Bước này sử dụng chế độ *-auto-approve* để đảm bảo quá trình chạy hoàn toàn tự động trong CI/CD. Điều kiện chạy apply phụ thuộc vào kết quả của bước plan, đảm bảo chỉ triển khai khi có thay đổi hoặc cần đồng bộ trạng thái.

Kết thúc workflow, hệ thống trả về hai output quan trọng là tên cluster EKS và namespace Kubernetes tương ứng với môi trường. Các giá trị này được sử dụng cho các pipeline tiếp theo như triển khai ứng dụng.

## 2.11. Triển khai ứng dụng

Khi được gọi, workflow nhận các tham số đầu vào quan trọng như tên image Docker (*full\_image\_name*), namespace Kubernetes, tên cluster EKS, vùng AWS và môi trường triển khai. Các tham số này giúp workflow có thể linh hoạt triển khai cùng một ứng dụng lên nhiều môi trường khác nhau như dev, staging hoặc production.

Trong phần cấu hình ban đầu, workflow thiết lập các biến môi trường để sử dụng xuyên suốt các bước triển khai. Những biến như `AWS_REGION`, `K8S_NAMESPACE`, `cluster_name` và `IMAGE_NAME` được ánh xạ trực tiếp từ input, giúp đảm bảo tính nhất quán giữa các bước. Đồng thời, workflow cũng khai báo quyền truy cập cần thiết, bao gồm quyền ghi nội dung repository, quyền sử dụng token định danh (id-token) và quyền đọc packages, nhằm đảm bảo runner có đủ quyền để tương tác với AWS và registry chứa image.

Bước đầu tiên trong job triển khai là checkout source code từ repository GitHub. Điều này giúp workflow truy cập được toàn bộ manifest Kubernetes nằm trong thư mục `k8s/`. Sau đó, workflow cấu hình AWS credentials thông qua action chính thức của AWS, sử dụng access key và secret key được lưu trong GitHub Secrets. Bước này đóng vai trò quan trọng vì nó cho phép GitHub Actions xác thực và thao tác với các dịch vụ AWS, đặc biệt là EKS.

Tiếp theo, workflow tiến hành cài đặt AWS CLI phiên bản mới nhất và công cụ kubectl. AWS CLI được dùng để giao tiếp với EKS và cập nhật kubeconfig, trong khi kubectl là công cụ chính để quản lý tài nguyên Kubernetes. Sau khi cài đặt xong, workflow thực hiện lệnh

```
aws eks update-kubeconfig
```

Bảng 6: Lệnh kết nối đến cluster

Lệnh được dùng để thiết lập kết nối đến cluster EKS. Lệnh này tạo ra cấu hình truy cập Kubernetes, giúp kubectl có thể gửi lệnh trực tiếp đến cluster tương ứng.

Khi đã kết nối thành công, workflow kiểm tra và tạo namespace Kubernetes nếu namespace chưa tồn tại. Điều này đảm bảo môi trường triển khai luôn sẵn sàng và tách biệt giữa các ứng dụng hoặc môi trường khác nhau. Nếu namespace đã tồn tại, workflow sẽ bỏ qua bước tạo mới và tiếp tục quá trình deploy.

Ở bước triển khai ứng dụng, workflow sử dụng lệnh `sed` để thay thế các placeholder trong file manifest Kubernetes. Cụ thể, các giá trị như namespace và image Docker được inject trực tiếp vào file `app-deploy.yaml`. Sau khi cập nhật xong, toàn bộ tài nguyên trong thư mục `k8s/` được áp dụng lên cluster bằng lệnh `kubectl apply`. Đồng

thời, workflow cũng kiểm tra danh sách pods để xác nhận rằng tài nguyên đã được tạo thành công.

```

---
# 1. DEPLOYMENT CHO JUICE SHOP
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-deployment
  namespace: "{{k8s_namespace}}"
  labels:
    app: app-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: app-deployment
  template:
    metadata:
      labels:
        app: app-deployment
    spec:
      containers:
        - name: app-deployment
          image: "{{app_image}}"
          imagePullPolicy: Always
          ports:
            - containerPort: 3000
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
---
# 2. SERVICE XUẤT PORT 80 CHO DAST SCAN
apiVersion: v1
kind: Service
metadata:
  name: app-deployment-service
  namespace: "{{k8s_namespace}}"
spec:
  type: LoadBalancer
  selector:
    app: app-deployment
  ports:
    - protocol: TCP
      port: 80 # Mở port 80 thẳng ra Internet
      targetPort: 3000 # Port thực tế của container

```

Bảng 7: Nội dung cấu hình trong file app-deploy.yaml

Sau khi deploy, workflow thực hiện bước xác minh trạng thái bằng cách theo dõi quá trình rollout của deployment. Lệnh *kubectl rollout status* được sử dụng để đảm bảo

rằng các pod mới đã được tạo thành công và ứng dụng đã sẵn sàng hoạt động trong thời gian giới hạn 120 giây. Nếu rollout thất bại hoặc quá thời gian, bước này sẽ báo lỗi và workflow sẽ dừng lại.

Cuối cùng, workflow lấy thông tin LoadBalancer từ Kubernetes service để xác định URL public của ứng dụng. Hostname được trích xuất từ service và sau đó được xử lý để đảm bảo định dạng URL chính xác. Kết quả URL này được xuất ra dưới dạng output của job, giúp các workflow khác hoặc người dùng có thể truy cập trực tiếp vào ứng dụng sau khi triển khai thành công.

Hiện tại, trong đồ án này quy trình triển khai ứng dụng lên môi trường Staging và Production có các bước giống nhau, chỉ khác nhau ở cấu hình hạ tầng mạng ở hai môi trường.

## 2.12. DAST

Workflow được thiết kế để có thể chạy theo ba cơ chế: chạy định kỳ theo lịch cron mỗi tuần. Điều này giúp đảm bảo việc kiểm thử bảo mật có thể diễn ra liên tục trong cả giai đoạn phát triển lẫn vận hành.

Pipeline thực hiện quá trình xác thực ứng dụng bằng cách tạo user test và lấy JWT token. Cụ thể, nó gửi request tạo tài khoản qua API `/api/Users`, sau đó đăng nhập qua `/rest/user/login` để nhận về JWT. Token này được trích xuất bằng `jq`, đồng thời được mask để tránh lộ thông tin nhạy cảm trong log CI/CD. JWT sau đó được đưa vào biến môi trường để phục vụ cho quá trình scan authenticated API.

Sau khi có token, pipeline tiến hành chạy active scan bằng action `zapproxy/action-full-scan`. Trong bước này, ZAP thực hiện quét toàn bộ ứng dụng với chế độ active attack (-a), đồng thời xuất kết quả dạng JSON (-j). JWT được inject vào header Authorization thông qua cơ chế replacer, giúp ZAP có thể truy cập các endpoint yêu cầu đăng nhập. Kết quả scan được xuất ra dưới dạng file HTML và JSON để phục vụ phân tích.

Kết quả scan sau đó được lưu trữ dưới dạng artifact trong GitHub Actions, bao gồm báo cáo HTML và JSON. Việc lưu trữ này cho phép đội ngũ bảo mật hoặc phát triển có thể tải về để phân tích chi tiết các lỗ hổng được phát hiện, đồng thời phục vụ audit hoặc điều tra sau này.

Pipeline tiếp tục thực hiện bước phân tích kết quả bằng cách đọc file JSON và lọc các lỗ hổng có mức độ High (*riskcode* = 3). Nếu phát hiện lỗ hổng nghiêm trọng, pipeline sẽ tự động tạo GitHub Issue chứa danh sách các vulnerability tiêu biểu, bao gồm tên lỗ hổng, mức độ rủi ro, CWE ID và endpoint bị ảnh hưởng. Sau đó workflow sẽ chủ động fail bằng *exit 1* để chặn quá trình promotion hoặc deployment, đảm bảo nguyên tắc security gate trong CI/CD.

Song song với đó, pipeline còn gửi metric về hệ thống giám sát thông qua Prometheus remote write tới Grafana. Metric chính là số lượng lỗ hổng được phát hiện trong lần scan, giúp xây dựng dashboard theo dõi xu hướng bảo mật theo thời gian.

Ngoài ra, toàn bộ kết quả scan cũng có thể được đẩy sang hệ thống log tập trung Grafana Loki. Dữ liệu JSON của ZAP được đóng gói thành log stream và gắn metadata như repository, workflow và loại scan, phục vụ việc truy vết và phân tích tập trung.

Cuối cùng, pipeline sử dụng một file rules để loại bỏ các cảnh báo không cần thiết hoặc nằm ngoài phạm vi kiểm thử, như các header thiếu không mang tính exploit cao hoặc các cảnh báo kỹ thuật mức thấp. Điều này giúp giảm nhiễu và tập trung vào các lỗ hổng có khả năng khai thác thực tế.

```
10109 IGNORE (Modern Web Application)
10035 IGNORE (Strict-Transport-Security Header Not Set)
10098 IGNORE (Cross-Domain Misconfiguration)
10017 IGNORE (Cross-Domain JavaScript Source File Inclusion)
10096 IGNORE (Timestamp Disclosure - Unix)
10015 IGNORE (Incomplete or No Cache-control and Pragma HTTP Header Set)
10038 IGNORE (Content Security Policy (CSP) Header Not Set)
10099 IGNORE (Source Code Disclosure - Java)
10027 IGNORE (Information Disclosure - Suspicious Comments)
10094 IGNORE (Base64 Disclosure)
10063 IGNORE (Feature Policy Header Not Set)
10049 IGNORE (Storable but Non-Cacheable Content)
10049 IGNORE (Non-Storable Content)
10110 IGNORE (Dangerous JS Functions)
90004 IGNORE (Insufficient Site Isolation Against Spectre Vulnerability)
90005 IGNORE (Sec-Fetch-Dest Header is Missing)
90005 IGNORE (Sec-Fetch-Mode Header is Missing)
90005 IGNORE (Sec-Fetch-Site Header is Missing)
90005 IGNORE (Sec-Fetch-User Header is Missing)
```

Bảng 8: Các rule được cấu hình

### 2.13. Policy Gate

Mục tiêu chính là đảm bảo mọi container image được deploy vào cluster đều phải được xác thực về chữ ký số và SBOM trước khi được phép chạy, từ đó giảm thiểu rủi ro supply chain attack.

Trong kiến trúc tổng thể, *OPA Gatekeeper* đóng vai trò là admission controller trong Kubernetes, intercept mọi request tạo tài nguyên như Pod, Deployment, Job, CronJob. Khi một workload được gửi lên cluster, Gatekeeper sẽ kích hoạt các policy viết bằng Rego để kiểm tra tính tuân thủ. Thay vì tự xử lý logic xác thực phức tạp trong Rego, hệ thống sử dụng *External Data Provider*, cho phép gọi ra một service bên ngoài để thực hiện xác thực chữ ký image bằng Cosign.

Thành phần *Cosign Gatekeeper Provider* được triển khai dưới dạng một HTTP service chạy trong namespace *cosign-system*. Service này nhận request từ Gatekeeper thông qua endpoint */validate*, sau đó thực thi hai bước kiểm tra chính. Đầu tiên, nó sử dụng lệnh *cosign verify* để xác thực chữ ký của container image dựa trên public key được mount từ Kubernetes Secret. Nếu chữ ký không hợp lệ hoặc không tồn tại, image sẽ bị đánh dấu không hợp lệ. Nếu vượt qua bước này, hệ thống tiếp tục thực hiện kiểm tra SBOM bằng *cosign verify-attestation* với định dạng SPDX. Điều này đảm bảo image không chỉ được ký mà còn có đầy đủ thông tin phụ thuộc phần mềm.

Trong tầng policy, hệ thống sử dụng *ConstraintTemplate: K8sRequiredImageSignatures* để định nghĩa logic kiểm soát. Rego policy được thiết kế để lấy danh sách image từ các workload khác nhau như Pod hoặc Deployment, sau đó gọi hàm *external\_data()* để truy vấn kết quả từ provider. Policy xử lý nhiều tình huống lỗi khác nhau như provider trả lỗi, system error, thiếu response hoặc image chưa được verify. Mỗi trường hợp đều dẫn đến một violation rõ ràng nhằm đảm bảo tính minh bạch trong lý do từ chối.

Constraint *require-signed-project-images* được áp dụng theo phạm vi giới hạn, chỉ enforce trong namespace *isolated-staging* và chỉ áp dụng cho các image thuộc prefix *ghcr.io/nt524/secure-ci-cd-pipeline*. Điều này giúp tránh ảnh hưởng tới các system workload của Kubernetes hoặc AWS, đồng thời đảm bảo policy chỉ tác động đến các image thuộc hệ thống CI/CD nội bộ.

Trong pipeline CI/CD sử dụng GitHub Actions, toàn bộ hạ tầng Gatekeeper và provider được tự động triển khai. Pipeline thực hiện việc cấu hình AWS credentials, cập nhật kubeconfig đến EKS cluster, sau đó cài đặt Gatekeeper và bật tính năng external data. Tiếp theo, pipeline đồng bộ các secret quan trọng bao gồm Cosign public key và credentials để truy cập GitHub Container Registry. Sau đó, provider image được build và push lên GHCR trước khi được deploy vào cluster.

Một điểm quan trọng trong pipeline là bước quản lý trạng thái policy. Trước khi áp dụng constraint mới, hệ thống chủ động xóa các policy cũ để tránh deadlock, sau đó deploy lại provider và chờ hệ thống ổn định. Khi provider đã sẵn sàng, ConstraintTemplate và Constraint được áp dụng lại để kích hoạt cơ chế kiểm soát admission một cách an toàn.

Về mặt bảo mật, thiết kế này tạo ra một lớp kiểm soát mạnh ở tầng admission, đảm bảo chỉ những image đã được ký và có SBOM mới được phép chạy trong cluster. Đồng thời, việc tách provider ra khỏi Gatekeeper giúp hệ thống linh hoạt hơn, dễ mở rộng logic xác thực mà không cần thay đổi policy Rego. Tuy nhiên, kiến trúc này cũng phụ thuộc vào availability của provider service, do đó cần cân nhắc thêm cơ chế HA hoặc fallback để tránh ảnh hưởng đến quá trình deploy khi provider gặp sự cố.

# Chương IV: KẾT QUẢ TRIỂN KHAI

## 1. Pre-commit

Run pre-commit hooks3885

```
1 ▶ Run pre-commit/action@v3.0.1
11 ▶ Run python -m pip install pre-commit
21 Collecting pre-commit
22   Downloading pre_commit-4.6.0-py2.py3-none-any.whl.metadata (1.2 kB)
23 Collecting cfgv>=2.0.0 (from pre-commit)
24   Downloading cfgv-3.5.0-py2.py3-none-any.whl.metadata (8.9 kB)
25 Collecting identify>=1.0.0 (from pre-commit)
26   Downloading identify-2.6.19-py2.py3-none-any.whl.metadata (4.4 kB)
27 Collecting nodeenv>=0.11.1 (from pre-commit)
28   Downloading nodeenv-1.10.0-py2.py3-none-any.whl.metadata (24 kB)
29 Collecting pyyaml>=5.1 (from pre-commit)
30   Downloading pyyaml-6.0.3-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.4 kB)
31 Collecting virtualenv>=20.10.0 (from pre-commit)
32   Downloading virtualenv-21.5.1-py3-none-any.whl.metadata (3.4 kB)
33 Collecting distlib<1,>=0.3.7 (from virtualenv>=20.10.0->pre-commit)
34   Downloading distlib-0.4.3-py2.py3-none-any.whl.metadata (5.3 kB)
35 Collecting filelock<4,>=3.24.2 (from virtualenv>=20.10.0->pre-commit)
36   Downloading filelock-3.29.4-py3-none-any.whl.metadata (2.0 kB)
37 Collecting platformdirs<5,>=3.9.1 (from virtualenv>=20.10.0->pre-commit)
38   Downloading platformdirs-4.10.0-py3-none-any.whl.metadata (5.5 kB)
39 Collecting python-discovery>=1.4.2 (from virtualenv>=20.10.0->pre-commit)
40   Downloading python_discovery-1.4.2-py3-none-any.whl.metadata (5.6 kB)
41 Downloading pre_commit-4.6.0-py2.py3-none-any.whl (226 kB)
42 Downloading cfgv-3.5.0-py2.py3-none-any.whl (7.4 kB)
43 Downloading identify-2.6.19-py2.py3-none-any.whl (99 kB)
44 Downloading nodeenv-1.10.0-py2.py3-none-any.whl (23 kB)
45 Downloading pyyaml-6.0.3-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (806 kB)
46   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 806.6/806.6 kB 65.5 MB/s 0:00:00
47 Downloading virtualenv-21.5.1-py3-none-any.whl (4.6 MB)
48   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.6/4.6 MB 152.0 MB/s 0:00:00
49 Downloading distlib-0.4.3-py2.py3-none-any.whl (470 kB)
50 Downloading filelock-3.29.4-py3-none-any.whl (42 kB)
51 Downloading platformdirs-4.10.0-py3-none-any.whl (22 kB)
52 Downloading python_discovery-1.4.2-py3-none-any.whl (33 kB)
```

Hệ thống ghi nhận một số công cụ quét tĩnh đã hoàn thành xuất sắc nhiệm vụ và đạt trạng thái an toàn tuyệt đối mà không phát hiện hành vi vi phạm nào. Cụ thể, cấu hình hạ tầng dưới dạng mã (IaC) được bảo vệ nghiêm ngặt thông qua công cụ *TFSec*. Song song với đó, các quy định về việc kiểm soát dung lượng tệp tin thêm mới và cơ chế rà soát mã khóa bí mật ghi trực tiếp đều kết thúc thành công, đảm bảo không có mã khóa thô hay tệp tin quá tải nào lọt vào hệ thống.

Dưới đây là bảng tổng hợp tất cả các lỗi xảy ra trong quá trình chạy pre-commit, phân loại rõ ràng giữa các lỗi đã được tự động sửa đổi và các lỗi nghiêm trọng bắt buộc phải can thiệp thủ công:

| STT | Tên Công Cụ<br>(Hook Name)  | Hook ID             | Mã thoát (Exit Code) |
|-----|-----------------------------|---------------------|----------------------|
| 1   | trim trailing<br>whitespace | trailing-whitespace | 1 (Failed)           |

|   |                  |                   |            |
|---|------------------|-------------------|------------|
| 2 | fix end of files | end-of-file-fixer | 1 (Failed) |
| 3 | check yaml       | check-yaml        | 1 (Failed) |
| 4 | Detect secrets   | detect-secrets    | 2 (Failed) |
| 5 | semgrep          | semgrep           | 1 (Failed) |

*Bảng 9: Bảng tổng hợp lỗi được quét ở pre-commit*

Do sự xuất hiện của các lỗi cấu hình nghiêm trọng (YAML) và lỗi an toàn mã nguồn (Semgrep), hệ thống đã kích hoạt cơ chế bắt buộc hành động (MANDATORY): Trả về mã thoát lỗi để đánh dấu toàn bộ Workflow là Thất bại. Việc này đồng nghĩa với việc chặn đứng hoàn toàn mọi tiến trình triển khai (deployment) hoặc chuyển giao mã nguồn lên các nhánh/môi trường cao hơn (promotion) nhằm đảm bảo an toàn cho sản phẩm.

Bên cạnh đó, hệ thống cũng ghi nhận một lỗi phụ phát sinh ở cuối tiến trình: do workflow thất bại, các tác vụ tương tác API với GitHub như tự động gắn nhãn (labels), tạo bình luận giải thích lỗi (comment) hoặc tự động đóng các Pull Request (PR) không hợp lệ đều không thể thực hiện được. Khuyến nghị ngay lúc này là đội ngũ lập trình viên cần chỉnh sửa thủ công các tệp YAML bị báo lỗi, kiểm tra lại tham số của hook detect-secrets, và xử lý dứt điểm các cảnh báo bảo mật từ Semgrep để giải phóng luồng tích hợp liên tục CI/CD.

## 2. Secret scanning

| STT | Tên Tệp (File Path) | Dòng | Loại Secret (Detector Name) | Trạng thái Xác minh |
|-----|---------------------|------|-----------------------------|---------------------|
| 1   | /pwd/.git/config    | 12   | Github (Personal            | Chưa xác            |

|   |                                         |     |                                |                       |
|---|-----------------------------------------|-----|--------------------------------|-----------------------|
|   |                                         |     | Access Token)                  | minh (false)          |
| 2 | /pwd/app/test/api/userApiSpec.ts        | 306 | JWT (JSON Web Token)           | Chưa xác minh (false) |
| 3 | /pwd/app/test/api/userApiSpec.ts        | 302 | JWT (JSON Web Token)           | Chưa xác minh (false) |
| 4 | /pwd/app/test/api/userApiSpec.ts        | 304 | JWT (JSON Web Token)           | Chưa xác minh (false) |
| 5 | /pwd/app/test/server/currentUserSpec.ts | 33  | JWT (JSON Web Token)           | Chưa xác minh (false) |
| 6 | /pwd/app/lib/insecurity.ts              | 23  | PrivateKey (Khóa bảo mật RSA)  | Chưa xác minh (false) |
| 7 | /pwd/trufflehog-report.json             | 1   | Github (Personal Access Token) | Chưa xác minh (false) |

|   |                             |   |                      |                                        |
|---|-----------------------------|---|----------------------|----------------------------------------|
| 8 | /pwd/trufflehog-report.json | 2 | JWT (JSON Web Token) | Chưa xác minh (false)                  |
| 9 | /pwd/trufflehog-report.json | 5 | JWT (JSON Web Token) | Chưa xác minh (false - dữ liệu bị cắt) |

*Bảng 10: Bảng thống kê chi tiết các cảnh báo ở bước Secret scanning*

Các cảnh báo từ số **7 đến 9** được phát hiện ngay bên trong tệp trufflehog-report.json. Điều này thường xảy ra khi công cụ quét chính tệp báo cáo mà nó (hoặc một tiến trình trước đó) vừa tạo ra trong không gian làm việc.

Phần lớn các cảnh báo liên quan đến **JWT (JSON Web Token)** nằm trong các tệp mã nguồn phục vụ mục đích kiểm thử (test) như userApiSpec.ts và currentUserSpec.ts.

Đặc biệt cần chú ý đến cảnh báo số **6** tại tệp insecurity.ts, nơi phát hiện một đoạn mã giống khóa riêng tư RSA (RSA Private Key).

Pipeline đã cấu hình lệnh Docker để quét toàn bộ hệ thống tệp và xuất kết quả định dạng JSON trực tiếp ra tệp trufflehog-report.json. Quá trình quét mã nguồn diễn ra với tốc độ rất nhanh, hoàn thành trong khoảng 5.2 giây (chính xác là 5.204521672s). Trong khoảng thời gian ngắn đó, TruffleHog đã phân tích tổng cộng 7,295 khối dữ liệu (chunks) với tổng khối lượng tương đương 84,835,500 byte.

Về số lượng rủi ro được phát hiện, báo cáo của TruffleHog cho thấy hệ thống an toàn ở mức độ xác thực, không phát hiện bất kỳ bí mật nào bị lộ (0 verified secrets). Tuy nhiên, công cụ quét cũng đã đánh dấu cảnh báo về 9 bí mật chưa được xác minh (9 unverified secrets). Đây là những chuỗi dữ liệu có cấu trúc giống thông tin nhạy cảm,

yêu cầu đội ngũ phát triển (developers) hoặc kỹ sư bảo mật xem xét thủ công để loại trừ các trường hợp dương tính giả (false positives).

### 3. IaC scanning:

| STT | Công cụ quét    | Tệp tin (File)         | Nội dung cảnh báo / Lỗi                                                           |
|-----|-----------------|------------------------|-----------------------------------------------------------------------------------|
| 1   | <b>Checkov</b>  | terraform/main.tf      | Module vpc và eks không sử dụng Git commit hash ở phần source (Mã lỗi: CKV_TF_1). |
| 2   | <b>Conftest</b> | terraform/main.tf      | CẢNH BÁO: EKS cluster không được bật public endpoint trong quét tĩnh CI.          |
| 3   | <b>Conftest</b> | terraform/providers.tf | CẢNH BÁO: AWS provider phải có default_tags trong quét tĩnh Terraform.            |
| 4   | <b>Conftest</b> | k8s/app-deploy.yaml    | CẢNH BÁO: Container app-deployment phải bật readOnlyRootFilesystem=true.          |
| 5   | <b>Conftest</b> | k8s/app-deploy.yaml    | CẢNH BÁO: Container app-deployment phải drop toàn bộ capabilities.                |

*Bảng 11: Bảng tổng hợp cảnh báo từ bước IaC scanning*

Đánh giá chung cho thấy hệ thống hạ tầng hiện tại đã đáp ứng tốt nhiều tiêu chuẩn bảo mật nền tảng như việc không để lộ thông tin đăng nhập quan trọng và áp dụng tốt các chính sách hạn chế đặc quyền container. Tuy nhiên, hệ thống vẫn ghi nhận một số lỗi và cảnh báo có mức độ ảnh hưởng đáng kể liên quan đến việc quản lý chuỗi cung ứng của các module và cấu hình an toàn mạng cần được khắc phục kịp thời.

Phân tích chi tiết từ công cụ Checkov đối với mã nguồn Terraform cho thấy một lỗ hổng đáng chú ý trong tệp tin terraform/main.tf, nơi các module vpc và eks bị đánh dấu là không đạt tiêu chuẩn do khai báo nguồn (source) không đi kèm mã băm Git commit cụ thể theo mã lỗi CKV\_TF\_1. Việc chỉ định nguồn module thông qua thẻ phiên bản thông thường dù đã giúp vượt qua bài kiểm tra CKV\_TF\_2 nhưng vẫn chưa tối ưu, vì nó tiềm ẩn nguy cơ bị thao túng hoặc thay đổi ngoài tầm kiểm soát từ phía nhà cung cấp bên thứ ba. Điểm sáng trong cấu hình hạ tầng là tệp tin terraform/providers.tf đã vượt qua bài kiểm tra bảo mật nghiêm ngặt CKV\_AWS\_41, bảo đảm rằng không có bất kỳ thông tin đăng nhập nhạy cảm nào như AWS access key hay secret key bị mã hóa cứng trực tiếp trong mã nguồn.

Đối với cấu hình môi trường container, Checkov đã ghi nhận kết quả rất tích cực khi các tệp cấu hình đạt tính tuân thủ cao đối với các quy chuẩn bảo mật tiêu chuẩn. Cụ thể, tệp định nghĩa ứng dụng Kubernetes k8s/app-deploy.yaml đã vượt qua các bài kiểm tra quan trọng như không chạy container dưới quyền đặc quyền (privileged) thông qua mã CKV\_K8S\_16, không kích hoạt chế độ xác thực lỏng lẻo AlwaysAllow thông qua mã CKV\_K8S\_74, và cấu hình các chứng chỉ TLS hoàn toàn hợp lệ. Thêm vào đó, tệp cấu hình /app/Dockerfile cũng được xác nhận là an toàn khi tuân thủ tốt các quy tắc không tắt tính năng xác thực chứng chỉ an toàn khi tải gói bằng các công cụ như wget, npm, node hay apk, đồng thời phân quyền thư mục logs một cách hợp lý và không sử dụng các lệnh leo thang đặc quyền không an toàn.

Song song với Checkov, công cụ Conftest đã thực thi các bộ quy tắc chính sách nội bộ và phát hiện ra một số cảnh báo quan trọng đối với các tệp cấu hình hiện tại. Tại tệp terraform/main.tf, mặc dù hệ thống ghi nhận 6 kiểm tra thành công, Conftest vẫn đưa ra cảnh báo về việc cụm EKS cluster không được bật public endpoint trong quá trình quét tĩnh CI để hạn chế các rủi ro tiếp xúc mạng bên ngoài. Đối với tệp terraform/providers.tf, công cụ yêu cầu nhà phát triển phải bổ sung thuộc tính default\_tags vào block khai báo AWS provider để tối ưu hóa việc gắn nhãn quản lý tài nguyên. Đáng chú ý, tệp cấu hình triển khai ứng dụng k8s/app-deploy.yaml dù đạt 10 kiểm tra thành công nhưng vẫn bị cảnh báo ở hai điểm, yêu cầu container app-deployment phải được thiết lập hệ thống tệp tin chỉ đọc bằng tham số

`readOnlyRootFilesystem=true` và phải loại bỏ hoàn toàn các quyền hạn hệ thống không cần thiết bằng cách drop toàn bộ capabilities. Ngược lại, các tệp cấu hình phụ trợ như `terraform/outputs.tf` và `terraform/variables.tf` hoàn toàn tuân thủ chính sách khi vượt qua toàn bộ các lượt kiểm tra mà không phát sinh bất kỳ cảnh báo nào.

#### 4. Dependency scanning

Trong quá trình quét, Dependabot đã phát hiện hàng loạt thư viện có chứa lỗ hổng bảo mật và yêu cầu các bản vá. Cụ thể, danh sách các thư viện bị ảnh hưởng bao gồm: `@cyclonedx/cyclonedx-npm` (từ bản 2.1.0 đến dưới 5.0.0), `multer` (phát hiện nhiều lỗ hổng ở các bản dưới 2.2.0, bao gồm cả các bản alpha và bản 1.x), `js-yaml` (các phiên bản từ 4.1.1 trở xuống), `sanitize-html` (nhiều lỗ hổng rải rác từ bản 2.17.3 trở xuống đến các bản 1.x), `file-type` (các phiên bản từ 13.0.0 đến 21.3.1), `socket.io` (các bản dưới 2.5.0 và đoạn từ 3.0.0 đến dưới 4.6.2), `jsonwebtoken` (các bản dưới 9.0.0), `marsdb` (tất cả các bản từ 0.0.0 trở lên), và `express-jwt` (từ bản 5.3.3 trở xuống).

#### 5. Container scanning

| Mã cảnh báo<br>/ CVE  | Thành phần<br>(Package)   | Mức độ<br>(Severity) | Mô tả chi tiết                                                                                                                                                             | Phiên bản<br>khắc phục |
|-----------------------|---------------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>CVE-2019-10744</b> | <code>lodash@2.4.2</code> | <b>CRITICAL</b>      | Lỗ hổng Prototype pollution trong hàm <code>defaultsDeep</code> có thể bị lợi dụng để thêm hoặc sửa đổi các thuộc tính của <code>Object.prototype</code> , dẫn đến nguy cơ | Cập nhật lên 4.17.12   |

|                         |                   |                               |                                                                                                                                                                                                   |                                               |
|-------------------------|-------------------|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
|                         |                   |                               | thay đổi logic ứng dụng.                                                                                                                                                                          |                                               |
| <b>CVE-2025-65945</b>   | jws@0.2.6         | <b>HIGH</b>                   | Lỗi xác thực chữ ký bị thiết kế sai lệch trong thuật toán HS256. Kẻ tấn công có thể cung cấp dữ liệu giả mạo từ header hoặc payload của JSON Web Signature để vượt qua quá trình xác minh chữ ký. | Cập nhật lên 3.2.3 hoặc 4.0.1                 |
| <b>CVE-2016-1000223</b> | jws               | <b>HIGH</b>                   | Lỗi hỏng nghiêm trọng liên quan đến thư viện JSON Web Token (phát hiện trong Node.js).                                                                                                            | N/A                                           |
| <b>Secret Exposure</b>  | Mã nguồn ứng dụng | <b>HIGH</b> / <b>CRITICAL</b> | Lộ lọt trực tiếp nội dung bắt đầu bằng ----BEGIN RSA PRIVATE KEY----. Khóa bí mật này nằm ở offset 860 trong layer copy mã                                                                        | Cần xóa bỏ khóa khỏi source code / Dockerfile |

|  |  |  |                    |  |
|--|--|--|--------------------|--|
|  |  |  | nguồn /juice-shop. |  |
|--|--|--|--------------------|--|

Bảng 12: bảng tổng hợp kết quả ở bước container scanning

Trong quá trình đánh giá, hệ thống đã phát hiện ra nhiều điểm yếu bảo mật đáng chú ý nằm ở các gói thư viện Node.js (npm) được tích hợp trong ứng dụng. Các lỗ hổng này dao động ở mức độ nghiêm trọng từ CAO (HIGH) đến ĐẶC BIỆT NGHIÊM TRỌNG (CRITICAL). Nổi bật nhất là các lỗi liên quan đến cơ chế xác thực chữ ký số (JSON Web Signature) và lỗ hổng ô nhiễm nguyên mẫu (Prototype Pollution) có thể dẫn đến việc thay đổi luồng thực thi của ứng dụng.

Đặc biệt nghiêm trọng, công cụ cũng đã rà quét và phát hiện ra sự tồn tại của dữ liệu nhạy cảm (Secret Exposure) ngay bên trong image. Cụ thể, một khóa riêng tư (RSA PRIVATE KEY) đã bị lộ lọt trực tiếp ở layer tạo ra bởi câu lệnh `COPY --chown=65532:0 /juice-shop . # buildkit`.

## 6. DAST

| Tên cảnh báo<br>(Vulnerability) | Mức độ rủi ro          | Số lượng | Mô tả sự cố / Giải pháp đề xuất                                                                                                                                                                             |
|---------------------------------|------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>HTTP Only Site</b>           | Trung bình<br>(Medium) | 1        | <p><b>Mô tả:</b> Trang web hiện chỉ được phục vụ qua giao thức HTTP không an toàn mà không có HTTPS.</p> <p><b>Đề xuất:</b> Cần triển khai chứng chỉ SSL/TLS và cấu hình chuyển hướng toàn bộ lưu lượng</p> |

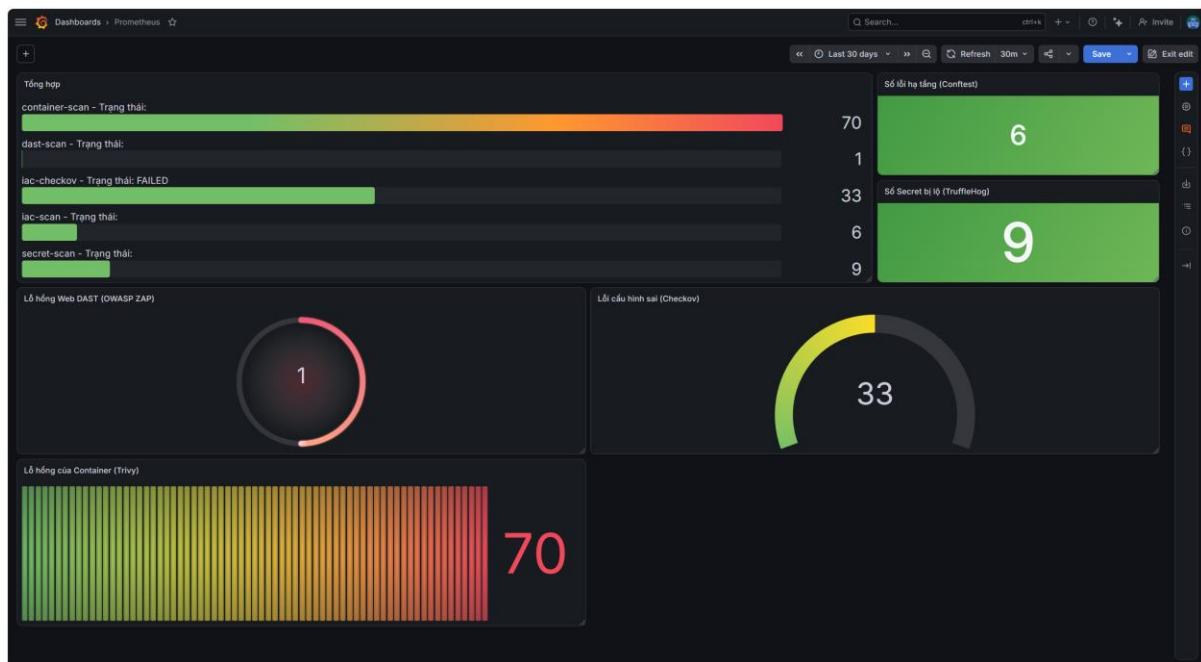
|                                         |                           |   |                                                                                                                                                                                                  |
|-----------------------------------------|---------------------------|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                         |                           |   | HTTP sang HTTPS.                                                                                                                                                                                 |
| <b>Non-Storable Content</b>             | Thông tin (Informational) | 5 | <b>Mô tả:</b> Phản hồi từ máy chủ ngăn cản nội dung được lưu trữ trong bộ nhớ đệm (cache) nhằm cải thiện hiệu suất, điều này có thể dẫn đến việc tăng tải cho hệ thống nếu không được kiểm soát. |
| <b>Sec-Fetch-Dest Header is Missing</b> | Thông tin (Informational) | 4 | <b>Giải pháp:</b> Cần đảm bảo rằng tiêu đề (header) Sec-Fetch-Dest được bổ sung đầy đủ vào các request headers để tăng cường bảo mật Fetch Metadata.                                             |
| <b>Sec-Fetch-Mode Header is Missing</b> | Thông tin (Informational) | 4 | <b>Giải pháp:</b> Tương tự, cần cấu hình thêm tiêu đề Sec-Fetch-Mode vào các HTTP request headers.                                                                                               |

|                                         |                           |   |                                                                                                                                      |
|-----------------------------------------|---------------------------|---|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sec-Fetch-Site Header is Missing</b> | Thông tin (Informational) | 4 | <b>Giải pháp:</b> Cần bổ sung tiêu đề bảo mật Sec-Fetch-Site vào các yêu cầu gửi đi để chống lại các cuộc tấn công giả mạo (CSRF).   |
| <b>Sec-Fetch-User Header is Missing</b> | Thông tin (Informational) | 4 | <b>Giải pháp:</b> Cần đảm bảo rằng tiêu đề Sec-Fetch-User được đưa vào các request do người dùng khởi tạo (user initiated requests). |

*Bảng 13: Bảng tổng hợp kết quả ở bước DAST*

Nhìn chung, hệ thống hiện tại không phát hiện ra các lỗ hổng mang tính chất nghiêm trọng (High) hay thấp (Low). Báo cáo ghi nhận tổng cộng có 1 cảnh báo ở mức độ rủi ro Trung bình (Medium) và 5 loại cảnh báo mang tính chất cung cấp thông tin (Informational). Hệ thống cũng không ghi nhận trường hợp dương tính giả (False Positives) nào trong suốt quá trình quét

## 7. Grafana Prometheus



Hệ thống đã triển khai thành công một Dashboard giám sát an ninh tập trung trên nền tảng **Grafana**, sử dụng dữ liệu trích xuất trực tiếp từ các công cụ quét tự động trong Pipeline DevSecOps qua **Prometheus**. Giao diện giám sát được cấu trúc trực quan với các nhóm Bảng chuyên biệt, cụ thể:

Các Bảng được triển khai và công dụng hiển thị:

### Bảng Tổng hợp trạng thái:

*Mô tả:* Nằm ở góc trên bên trái, thiết kế dưới dạng thanh đo tiến trình xếp chồng theo từng danh mục quét (container-scan, dast-scan, iac-checkov, iac-scan, secret-scan).

*Công dụng:* Giúp người quản trị so sánh tương quan trực quan giữa các thành phần, nhận diện ngay lập tức công cụ quét nào đang phát hiện nhiều vấn đề nhất thông qua hệ thống dải màu cảnh báo chuyển từ Xanh (An toàn) sang Đỏ (Nguy hiểm).

### Các Bảng số lượng tĩnh:

*Bảng Số lỗi hạ tầng (ConfTest):* Hiển thị dạng thẻ số lớn, ở góc trên bên phải. Công dụng phản ánh trực tiếp tổng số lượng các quy tắc chính sách hạ tầng nội bộ (Policy-as-Code) bị vi phạm trong các tệp cấu hình.

*Bảng Số Secret bị lộ (TruffleHog):* Thiết kế dạng thẻ số lớn đặt ngay phía dưới Bảng Conftest. Công dụng giám sát và cảnh báo nhanh số lượng thông tin nhạy cảm (như mật khẩu, key, token) vô tình bị rò rỉ trong lịch sử mã nguồn.

### **Các Bảng, Biểu đồ đo:**

*Bảng Lỗi hồng Web DAST (OWASP ZAP):* Sử dụng biểu đồ vòng Donut khép kín. Công dụng cô đọng và hiển thị mức độ nguy hại của các lỗ hồng ứng dụng động được phát hiện khi chạy thử nghiệm trên môi trường Staging.

*Bảng Lỗi cấu hình sai (Checkov):* Sử dụng biểu đồ đo dạng bán nguyệt (Gauge). Công dụng trực quan hóa mức độ nghiêm trọng của các lỗi an ninh hạ tầng dạng cấu hình sai so với bộ tiêu chuẩn bảo mật đám mây toàn cầu.

### **Bảng Lỗi hồng của Container (Trivy):**

*Mô tả:* Chiếm diện tích lớn ở góc dưới bên trái, thiết kế dạng các cột vạch màu san sát nhau tạo thành một dải quang phổ nhiệt chuyển từ sắc vàng sang đỏ đậm.

*Công dụng:* Chuyên biệt cho việc giám sát chuyên sâu, phản ánh tổng số lượng lỗi an ninh tích lũy tồn đọng trong thư viện hệ điều hành nền (Base Image) và các package mã nguồn của container.

**Dựa trên dữ liệu thu thập thực tế tại bảng, khung của Grafana Prometheus, hệ thống đánh giá các chỉ số:**

**Chỉ số Secret Exposure (TruffleHog):** Ghi nhận tổng số **9 lỗi**, cảnh báo mức độ rủi ro cao về việc lộ Private Key, API Tokens hoặc AWS Credentials trong mã nguồn lịch sử commit.

*Kết luận:* Nguy cơ cao bị lộ tài khoản đặc quyền, nhóm phát triển cần thu hồi khẩn cấp các token/key đã lộ và cấu hình chuyển đổi toàn bộ việc lưu trữ mật sang dịch vụ AWS Secrets Manager thay vì hardcoded.

**Chỉ số IaC Security (Checkov & Conftest):** Ghi nhận **33 lỗi** từ Checkov (cấu hình sai hạ tầng như mở cổng bừa bãi, không bật mã hóa ổ đĩa EBS) và **6 lỗi** từ Conftest (vi phạm chính sách nội bộ Policy-as-Code do nhóm tự định nghĩa).

*Kết luận:* Hạ tầng tính chưa tuân thủ tiêu chuẩn an toàn, yêu cầu kỹ sư hệ thống phải cập nhật lại file thiết kế để ngăn chặn rủi ro bị tấn công ngay từ bước thiết lập.

**Chỉ số Container Image Vulnerabilities (Trivy):** Ghi nhận **70 lỗ hổng**, phản ánh các thư viện base-image của OS (Debian) hoặc package Node.js đã cũ, chứa các mã CVE chưa được vá.

*Kết luận:* Bề mặt tấn công của container lớn do thư viện lỗi thời, cần nâng cấp ngay các package Node.js và chuyển dịch sang sử dụng các Base Image tinh gọn như Distroless hoặc Alpine để giảm tối đa mã CVE tồn đọng.

**Chỉ số Dynamic Application Security (OWASP ZAP DAST):** Ghi nhận **1 lỗ hổng**, biểu thị rủi ro xuất hiện khi ứng dụng thực tế khởi chạy trên môi trường Staging như lỗi HTTP Only Site hoặc thiếu các header bảo mật HTTP.

*Kết luận:* Ứng dụng dính kẻ hở ở tầng cấu hình dịch vụ web khi chạy thực tế, nhóm cần bổ sung ngay các header bảo mật HTTP bắt buộc và thiết lập chứng chỉ SSL/HTTPS để mã hóa toàn vẹn kênh truyền, chống tấn công MitM.

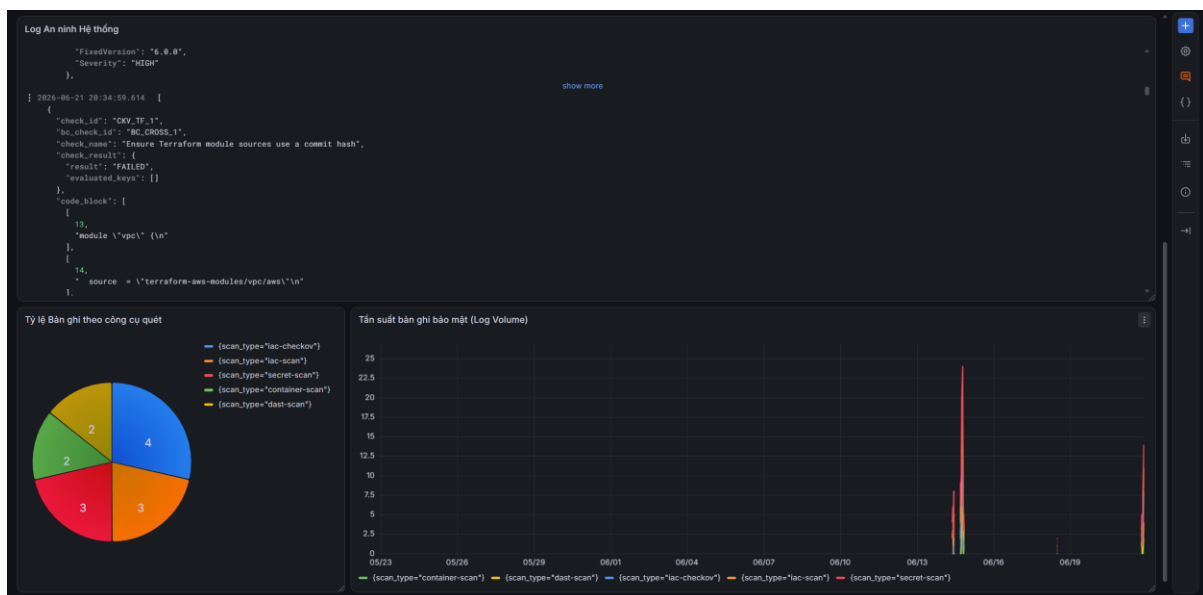
## 8. Grafana Loki

**Bảng tra cứu Lỗi hồng quét động (OWASP ZAP DAST)**

| labels                                         | Time | Line               | tsNs              | labelTypes                 | id                       | traceID | traceID (field) |
|------------------------------------------------|------|--------------------|-------------------|----------------------------|--------------------------|---------|-----------------|
| { "generated": "Sun, 14 J 2026-06-14 18:53:08" |      | @programName:"ZAP" | @version:"2.17.0" | { "generated": "P", "_pro" | 1781437988099114645_2683 |         |                 |
| { "generated": "Sun, 14 J 2026-06-14 19:37:24" |      | @programName:"ZAP" | @version:"2.17.0" | { "generated": "P", "_pro" | 1781440644464035942_b8f  |         |                 |
| { "generated": "Sun, 21 J 2026-06-21 18:49:34" |      | @programName:"ZAP" | @version:"2.17.0" | { "generated": "P", "_pro" | 1782042574320432741_489f |         |                 |
| { "generated": "Sun, 21 J 2026-06-21 20:05:20" |      | @programName:"ZAP" | @version:"2.17.0" | { "generated": "P", "_pro" | 1782047120820084792_1965 |         |                 |
| { "generated": "Sun, 21 J 2026-06-21 20:38:54" |      | @programName:"ZAP" | @version:"2.17.0" | { "generated": "P", "_pro" | 1782049134520661350_79e4 |         |                 |

**Metadata thông tin Repo & Hạ tầng (Target Information Metadata)**

| labels                                                | Time                | Line                                                                             | tsNs               | labelTypes                                     | id | traceID | traceID (field) |
|-------------------------------------------------------|---------------------|----------------------------------------------------------------------------------|--------------------|------------------------------------------------|----|---------|-----------------|
| known, "repo": "NT524/Secure"                         | 2026-06-14 08:38:41 | ainer app-deployment phải drop toàn bộ capabilities.", "me                       | 178140112115177472 | "S", "repo": "I", "scan_ty40112115177472_5980a |    |         |                 |
| "", "Redacted": "", "SourceID": "2026-06-14 08:38:44" |                     | "DecoderName": "PLAIN", "Verified": false, "VerificationFrom 1781401124787468680 |                    | system_file": "P", "Sourd01124787468680_bclb   |    |         |                 |
| "", "Redacted": "", "SourceID": "2026-06-14 08:45:54" |                     | "DecoderName": "PLAIN", "Verified": false, "VerificationFrom 1781401554388892748 |                    | system_file": "P", "Sourd01554388892748_bclb   |    |         |                 |
| known, "repo": "NT524/Secure"                         | 2026-06-14 08:46:04 | "namespace": "main", "successes": 7 }, { "filename": "k8:1781401564784820037"    |                    | "S", "repo": "I", "scan_ty401564784820037_826  |    |         |                 |



Hệ thống đã tích hợp và cấu hình thành công công cụ **Grafana Loki** đóng vai trò là giải pháp quản lý nhật ký tập trung. Toàn bộ dữ liệu nhật ký thô (Raw Logs) và siêu dữ liệu (Metadata) từ các công cụ quét an ninh trong pipeline CI/CD đã được cấu trúc hóa, chuyển đổi và đẩy thẳng về Loki làm kho lưu trữ lâu dài.

### Bảng Tra cứu Lỗi hồng quét động (OWASP ZAP DAST):

Mô tả: Nằm tại khu vực trung tâm, hiển thị danh sách nhật ký được phân tách chi tiết theo dạng bảng.

Công dụng: Cho phép kỹ sư an ninh lọc, truy vấn nhanh và xem trực tiếp toàn bộ dữ liệu phản hồi dạng cấu trúc của đợt kiểm thử động ứng dụng web.

### Bảng Metadata thông tin Repo & Hạ tầng:

Mô tả: Nằm ngay phía dưới bảng quét động, cấu hình để hứng các luồng log mang nhãn (scan\_type) đặc thù liên quan đến mã nguồn và tệp cấu hình.

Công dụng: Định vị chính xác đường dẫn tệp tin và số dòng code cụ thể xảy ra lỗi trong hạ tầng (ví dụ: file cấu hình .tf hoặc .yaml) mà không cần mở lại mã nguồn gốc.

### **Khung Log An ninh Hệ thống:**

Mô tả: Khung hiển thị log mở rộng định dạng văn bản JSON tường minh, hỗ trợ tự động thụt lề thụt dòng.

Công dụng: Giúp phân tích sâu vào chi tiết các thuộc tính của cảnh báo bảo mật như: mã định danh lỗ hổng quốc tế (CWEID, VulnerabilityID), độ tin cậy (confidence), giải pháp khắc phục khuyến nghị (solution), và thông tin registry.

### **Bảng Tỷ lệ Bản ghi theo công cụ quét:**

Mô tả: Biểu đồ hình tròn phân chia theo các lát màu sắc tương ứng với các nhãn dữ liệu của pipeline.

Công dụng: Trực quan hóa tỷ trọng log sinh ra từ các nguồn (iac-checkov, iac-scan, secret-scan, container-scan, dast-scan), giúp đánh giá phân hệ nào đang ghi nhận nhiều sự kiện hệ thống nhất.

### **Bảng Tần suất bản ghi bảo mật - Log Volume:**

Mô tả: Biểu đồ đường, cột theo trục thời gian tuyến tính thực tế.

Công dụng: Giám sát mật độ và chu kỳ kích hoạt của hệ thống quét, tạo điểm nhấn đột biến (Spike) trực quan vào những mốc thời gian pipeline CI/CD chạy lệnh push/pull.

**Dựa trên dữ liệu thu thập thực tế tại bảng, khung của Grafana Loki, hệ thống đánh giá các chỉ số:**

**Chỉ số quét động (DAST):** Ghi nhận tại Bảng Tra cứu Lỗ hổng quét động, hệ thống bắt được lỗ hổng HTTP Only Site (riskdesc: Medium, CWEID: 311) khi ứng dụng chạy trên AWS Endpoint do chưa cấu hình mã hóa SSL/HTTPS.

Kết luận: Cảnh báo ứng dụng dính rủi ro tấn công xen giữa (MitM) và lộ cookie, nhóm phát triển bắt buộc phải cấu hình bổ sung chứng chỉ SSL/HTTPS cho AWS Load Balancer và bật thuộc tính HttpOnly để vá triệt để lỗ hổng này.

**Chỉ số quét hạ tầng tĩnh (IaC Scan):** Ghi nhận tại Bảng Metadata thông tin Repo & Hạ tầng, hệ thống bắt được các trạng thái cấu hình FAILED từ file Kubernetes Manifest (như container chưa hạ quyền capabilities, chưa bật readOnlyRootFilesystem=true) và Terraform.

Kết luận: Cảnh báo tệp Manifest chưa tuân thủ các quy tắc an ninh tối thiểu (Security Context), cần rà soát lại cấu hình hạ tầng để cô lập quyền ghi của container, ngăn chặn hoàn toàn nguy cơ tấn công leo thang đặc quyền.

**Chỉ số tỷ trọng nguồn dữ liệu:** Ghi nhận tại Bảng Tỷ lệ Bản ghi, trực quan hóa phân phối tổng số lượng lỗi ghi vết gồm 4 bản ghi iac-checkov, 3 bản ghi iac-scan, 3 bản ghi secret-scan, 2 bản ghi container-scan và 2 bản ghi dast-scan.

Kết luận: Xác định các lỗi cấu hình sai hạ tầng tĩnh (IaC) đang chiếm tỷ trọng lớn nhất hệ thống; định hướng cho nhóm ưu tiên tập trung nhân lực tối ưu hóa file Terraform/K8s trước khi xử lý các cấu phần phụ trợ khác.

**Chỉ số biến động tần suất:** Ghi nhận tại Bảng Tần suất bản ghi bảo mật, đo lường dòng chảy dữ liệu với điểm spike đột biến vượt ngưỡng 22.5 bản ghi log tập trung cao độ vào giai đoạn ngày 14/06 và duy trì ghi vết đều đặn đến ngày 21/06.

Kết luận: Chứng minh hệ thống Loki vận hành ổn định, ghi vết liên tục không mất log, các điểm spike phản ánh chính xác chu kỳ push code mật độ cao của dự án và khẳng định khả năng xử lý luồng dữ liệu an ninh theo thời gian thực.

## 9. Minh chứng thực tế:

Link

demo:

[https://drive.google.com/drive/folders/1\\_SdNfvH4kJuqiYpiCTT2X5ctdW963tU9?usp=sharing](https://drive.google.com/drive/folders/1_SdNfvH4kJuqiYpiCTT2X5ctdW963tU9?usp=sharing)

Link repo: <https://github.com/NT524/Secure-CI-CD-pipeline.git>

Link web app staging: <http://a3d333440d8cb4dba93c7d8c6830af62-46359130.us-east-1.elb.amazonaws.com/>

Link web app prod: <http://af193c5e4d16746c3a33feb75977a129-1059821351.us-east-1.elb.amazonaws.com/>

## Chương V. KẾT LUẬN.

### 1. Kết quả đạt được của đồ án

Đồ án đã xây dựng và tích hợp thành công mô hình pipeline DevSecOps tự động hóa hoàn toàn luồng kiểm tra an ninh từ pha lập trình đến pha triển khai thực tế. Bằng việc áp dụng giải pháp Shift-Left, toàn bộ mã nguồn ứng dụng, tệp tin cấu hình hạ tầng tĩnh (IaC) và container đóng gói đều được quét tự động nhằm phát hiện sớm các rủi ro bảo mật như lộ dữ liệu nhạy cảm (Secrets), cấu hình sai thiết lập, hoặc lỗ hổng thư viện hệ điều hành. Hệ thống đã hiện thực hóa việc chuẩn hóa toàn bộ dữ liệu nhật ký phân tán từ các công cụ kiểm thử độc lập về cấu trúc tập trung, giúp giảm thiểu tối đa quy trình rà soát thủ công của đội ngũ vận hành. Dữ liệu an ninh bao gồm cả luồng quét động (DAST) ngoài môi trường Internet đều được tổng hợp đồng bộ theo thời gian thực, cung cấp góc nhìn toàn diện để kiểm soát chặt chẽ trạng thái an toàn của toàn bộ vòng đời sản phẩm.

### 2. Những hạn chế và sai sót gặp phải

Mặc dù hệ thống đã vận hành ổn định và đạt được các mục tiêu an ninh đề ra, đồ án vẫn khó tránh khỏi một số hạn chế mang tính khách quan và sai sót kỹ thuật phổ biến mà các hệ thống CI/CD tích hợp lần đầu thường gặp phải:

*Thiếu đồng bộ định dạng dữ liệu:* Cấu trúc log thô từ pipeline chưa được quy chuẩn hoàn toàn, gây lỗi phân tích cú pháp (JSONParserErr) cục bộ và làm mất một số siêu dữ liệu ở các lượt chạy phức tạp.

*Hạn chế về môi trường Multi-Cloud:* Giải pháp tự động hóa và các chính sách kiểm duyệt hiện tại mới chỉ được cấu hình, thử nghiệm độc lập trên nền tảng AWS, chưa mở rộng và đánh giá tính tương thích trên các dịch vụ đám mây khác.

### 3. Hướng phát triển và giải pháp khắc phục trong tương lai

Để khắc phục các điểm nghẽn nêu trên và nâng cao cho mô hình DevSecOps, hướng phát triển tiếp theo của đồ án sẽ tập trung vào các giải pháp cụ thể sau:

*Chuẩn hóa toàn diện luồng dữ liệu:* Đồng bộ flag đầu ra định dạng JSON cho mọi công cụ quét và bổ sung lớp validation trung gian để làm sạch chuỗi log, triệt tiêu lỗi phân tích cú pháp hệ thống.

*Mở rộng đa đám mây:* Chuẩn hóa mã nguồn Terraform để sẵn sàng mở rộng triển khai mô hình sang GCP hoặc Azure, hiện thực hóa giải pháp an ninh Multi-Cloud hoàn chỉnh.

## Tài liệu tham khảo

- [1] European Union Agency for Cybersecurity (ENISA), “Threat Landscape for Supply Chain Attacks,” ENISA Report, July 2021. [Trực tuyến]. Available: <https://www.enisa.europa.eu/publications/threat-landscape-for-supply-chain-attacks>.
- [2] D. Ohm, H. Plate, A. Sykosch và M. Meier, “Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks,” *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, pp. 23-43, 2020.
- [3] Mahmud, R. M. A. Hasan và M. A. A., “Towards automated security in DevOps: A comprehensive mapping study,” *IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC)*, pp. 251-256, 2018.
- [4] Viettel IDC, “DevOps là gì? Cách thức hoạt động của DevOps,” 26 11 2024. [Trực tuyến]. Available: <https://viettelidc.com.vn/tin-tuc/devops-la-gi>.

- [5] Amazon Web Services, “What is DevOps?,” [Trực tuyến]. Available: <https://aws.amazon.com/vi/devops/what-is-devops/>.
- [6] VNPT Cloud, “DevOps là gì? Toàn bộ kiến thức cần biết về DevOps cho người mới,” 2. [Trực tuyến]. Available: <https://cloud.vnpt.vn/blog/devops-la-gi-414>.
- [7] H. T. Thi, “Tìm hiểu về DevOps (Phần 1),” VIBLO, 14 06 2018. [Trực tuyến]. Available: <https://viblo.asia/p/tim-hieu-ve-devops-phan-1-ByEZk9Mo5Q0>.
- [8] Amzon Web Services, “What is DevSecOps?,” [Trực tuyến]. Available: <http://aws.amazon.com/vi/what-is/devsecops/>.
- [9] Microsoft, “DevSecOps là gì?,” [Trực tuyến]. Available: <https://www.microsoft.com/vi-vn/security/business/security-101/what-is-devsecops>.
- [10] VNPT Cloud, 15 04 2026. [Trực tuyến]. Available: <https://cloud.vnpt.vn/blog/ci-cd-la-gi-vai-tro-cua-ci-cd-trong-devops-hien-dai-340>.
- [11] L. M. Thiện và T. C. Thiện, “Xây dựng quy trình CI/CD mẫu cho ứng dụng web dựa trên microservices trên hệ thống cục bộ và AWS,” Khoa Mạng máy tính và Truyền thông - trường Đại học Công nghệ Thông tin, Đại học Quốc gia TP.HCM, 2024. [Trực tuyến]. Available: <https://nc.uit.edu.vn/do-an/xay-dung-quy-trinh-ci-cd-mau-cho-ung-dung-web-dua-tren-microservices-tren-he-thong-cuc-bo-va-aws>.
- [12] N. T. Hoa, “Shift Left Testing - Bí quyết cho Phần mềm thành công,” 27 02 2018. [Trực tuyến]. Available: <https://viblo.asia/p/shift-left-testing-bi-quyet-cho-phan-mem-thanh-cong-oOVIY14zl8W>.

- [13] ASIC CyberSecurity, “Shift Left Testing – Bí quyết cho Phần mềm thành công,” [Trực tuyến]. Available: <https://asiccybersecurity.vn/tin-tuc/shift-left-testing-bi-quyet-cho-phan-mem-thanh-cong/>.
- [14] Viện FMIT, “Shift Left Testing là gì - Kiểm thử dịch chuyển sang trái là gì,” [Trực tuyến]. Available: <https://fmit.vn/tu-dien-quan-ly/shift-left-testing-la-gi>.
- [15] VNPT Cloud, “Cloud Computing là gì? Tìm hiểu toàn tập về Cloud Computing,” 12 04 2022. [Trực tuyến]. Available: <https://cloud.vnpt.vn/blog/cloud-computing-la-gi-100>.
- [16] N. Ánh, “Cloud Computing là gì? Những điều cần biết về điện toán đám mây,” MISA, 28 09 2023. [Trực tuyến]. Available: <https://amis.misa.vn/110401/cloud-computing-la-gi/>.
- [17] Amazon Web Services, “What is cloud computing?,” [Trực tuyến]. Available: <https://aws.amazon.com/what-is-cloud-computing/>.
- [18] Accelario, “Production Environment,” [Trực tuyến]. Available: <https://accelario.com/glossary/production-environment/>.
- [19] P. H. Hoàng, “Giải ngố về môi trường (environment) – Một khái niệm/kỹ thuật cực hữu ích trong ngành lập trình,” [Trực tuyến]. Available: <https://toidicodedao.com/2019/07/02/environment-trong-lap-trinh/>.
- [20] LaunchDarkly, “What Is a Production Environment?,” 12 01 2023. [Trực tuyến]. Available: <https://launchdarkly.com/blog/what-is-a-production-environment/>.
- [21] Viện Công Nghệ Thông Tin Và Truyền Thông ITPlus, “NẮM RÕ VỀ MÔI TRƯỜNG LẬP TRÌNH QUA BÀI VIẾT SAU ĐÂY,” 22 07 2019.

- [Trực tuyến]. Available: <https://itplus.edu.vn/nam-ro-ve-moi-truong-lap-trinh-qua-bai-viet-sau-day-2396.html>.
- [22] S. J. Bigelow và A. S. Gillis, “staging environment,” 29 04 2024. [Trực tuyến]. Available: <https://www.techtarget.com/searchsoftwarequality/definition/staging-environment>.
- [23] Viettel, “Môi trường staging là gì? Vai trò của staging environment trong phát triển phần mềm,” 20 12 2025. [Trực tuyến]. Available: <http://viettelidc.com.vn/tin-tuc/moi-truong-staging-la-gi>.
- [24] Tempi, “Môi trường staging là gì? Toàn tập từ A-Z cho người mới bắt đầu,” [Trực tuyến]. Available: <https://tempi.vn/moi-truong-staging-la-gi-toan-tap-tu-a-z-cho-nguoi-moi-bat-dau>.
- [25] Tempi, “Môi Trường Staging Là Gì? Cách Thiết Lập Hiệu Quả,” [Trực tuyến]. Available: <https://tempi.vn/moi-truong-staging-la-gi-cach-thiet-lap-hieu-qua>.
- [26] DevOps VietNam, “Securing DevOps: Chiến lược bảo mật hiện đại cho hạ tầng Cloud và luồng CI/CD,” 24 08 2024. [Trực tuyến]. Available: <https://devops.vn/posts/securing-devops-review/>.
- [27] IBM, “What is DevOps monitoring?,” 12 02 2026. [Trực tuyến]. Available: <https://www.ibm.com/think/topics/devops-monitoring>.
- [28] DevOps VietNam, “DevSecOps Toàn Diện: Bảo Mật Từ Góc Nhìn Trong Phát Triển Phần Mềm Hiện Đại,” 14 07 2025. [Trực tuyến]. Available: <https://devops.vn/posts/devsecops-toan-dien-bao-mat-tu-goc-re-trong-phat-trien-phan-mem-hien-dai/>.

- [29] A. Bar, “What Is DevOps Secrets Management? Best Practices for Securing Your Pipelines,” Akeyless, 20 11 2025. [Trực tuyến]. Available: <https://www.akeyless.io/blog/secrets-management-for-devops-best-practices/>.
- [30] EVOTEK, “Quản lý Bí mật (Secrets Management) và QA: Ngăn chặn Rò rỉ Dữ liệu trong Kiểm thử,” 20 05 2025. [Trực tuyến]. Available: <https://tuyendung.evotek.vn/quan-ly-bi-mat-secrets-management-va-qa-ngan-chan-ro-ri-du-lieu-trong-kiem-thu/>.
- [31] GitHub, “Secret scanning,” [Trực tuyến]. Available: <https://docs.github.com/en/code-security/concepts/secret-security/secret-scanning>.
- [32] Viện FMIT, “Dependency scanning là gì - Quét phụ thuộc là gì,” [Trực tuyến]. Available: <https://fmit.vn/tu-dien-quan-ly/dependency-scanning-la-gi>.
- [33] Viện FMIT, “Dependency scan execution là gì? - Thực thi quét phụ thuộc là gì,” [Trực tuyến]. Available: <https://fmit.vn/tu-dien-quan-ly/dependency-scan-execution-la-gi>.
- [34] Amazon Web Services, “What is Infrastructure as Code?,” [Trực tuyến]. Available: <https://aws.amazon.com/what-is/iac/>.
- [35] VNPT Cloud, “Infrastructure as Code (IaC) là gì? Hiểu về cơ sở hạ tầng dưới dạng mã,” 13 05 2026. [Trực tuyến]. Available: <https://cloud.vnpt.vn/blog/infrastructure-as-code-iac-la-gi-351>.
- [36] Viện FMIT, “Quét hạ tầng dưới dạng mã là gì (Infrastructure as code scanning là gì),” [Trực tuyến]. Available: <https://fmit.vn/tu-dien-quan-ly/infrastructure-as-code-scanning-la-gi>.

- [37] SentinelOne, “What Is Infrastructure as Code (IaC) Scanning?,” 07 09 2025. [Trực tuyến]. Available: <https://www.sentinelone.com/cybersecurity-101/cloud-security/infrastructure-as-code-iac-scanning/>.
- [38] C. Queen, “IaC Scanning: Definition, Processes, and Technologies,” CrowdStrike, 08 05 2024. [Trực tuyến]. Available: <https://www.crowdstrike.com/en-us/cybersecurity-101/cloud-security/iac-scanning/>.
- [39] S. Phạm, “Container image là gì? Có gì khác so với Docker image?,” Bizfly Cloud, 10 04 2025. [Trực tuyến]. Available: <https://bizflycloud.vn/tin-tuc/container-image-la-gi-20250410151811468.htm>.
- [40] L. T. Xuyên, “Công nghệ Docker: Nền tảng Container hóa hiện đại,” FPT Aptech, [Trực tuyến]. Available: <https://aptech.fpt.edu.vn/cong-nghe-docker-nen-tang-container-hoa-hien-dai.html>.
- [41] A. Shah và J. P. P, “Container Security and How to Secure Containers in 2026,” ACCUKNOX, 11 12 2025. [Trực tuyến]. Available: <https://accuknox.com/blog/container-security>.
- [42] SentinelOne, “Container Vulnerability Management: Securing in 2026,” 06 05 2026. [Trực tuyến]. Available: <https://www.sentinelone.com/cybersecurity-101/cloud-security/container-vulnerability-management/>.
- [43] N. Ehrman, “Container Image Signing,” Wiz, [Trực tuyến]. Available: <https://www.wiz.io/academy/container-security/container-image-signing>.

- [44] Github, “What is a software bill of materials (SBOM)?,” 05 09 2025. [Trực tuyến]. Available: <https://github.com/resources/articles/what-is-an-sbom-software-bill-of-materials>.
  
- [45] T. Khan và M. Goodwin, “What is a software bill of materials (SBOM)?,” [Trực tuyến]. Available: <https://www.ibm.com/think/topics/sbom>.
  
- [46] Viettel, “Tự quản lý Docker image tại chỗ có an toàn? Cách kiểm soát và mã hóa image qua Container Registry,” 17 06 2025. [Trực tuyến]. Available: <https://viettelidc.com.vn/tin-tuc/tu-quan-ly-docker-image-tai-cho-co-an-toan-cach-kiem-soat-va-ma-hoa-image-qua-container-registry-4636>.
  
- [47] GreenNode, “Container Registry là gì? Tại sao doanh nghiệp nên sử dụng Container Registry?,” VNG Cloud, 24 02 2024. [Trực tuyến]. Available: <https://vngcloud.vn/vi/blog/what-is-a-container-registry>.
  
- [48] T. O. S. Team, “8 Container Security Best Practices for 2026,” Orca, 10 06 2026. [Trực tuyến]. Available: <https://orca.security/resources/blog/container-security-best-practices/>.
  
- [49] T. T. T. Vi, “Tổng quan về Kiểm thử bảo mật ứng dụng động (DAST),” SMOne, 11 10 2024. [Trực tuyến]. Available: <https://www.smone.vn/tong-quan-ve-kiem-thu-bao-mat-ung-dung-dong-dast/>.
  
- [50] Plexicus, “DAST là gì (Kiểm tra bảo mật ứng dụng động),” [Trực tuyến]. Available: <https://www.plexicus.ai/vi/glossary/dast/>.
  
- [51] D. V. Concepts, “DAST là gì? Dynamic Application Security Testing và cách dùng đúng trong DevOps,” DevOps VietNam, 18 03 2018.

- [Trực tuyến]. Available: <https://devops.vn/posts/dast-la-gi-cach-dung-trong-devops/>.
- [52] DevOps VietNam Research , “DevSecOps Toàn Diện: Bảo Mật Từ Gốc Rễ Trong Phát Triển Phần Mềm Hiện Đại,” DevOps VietNam, 14 07 2025. [Trực tuyến]. Available: <https://devops.vn/posts/devsecops-toan-dien-bao-mat-tu-goc-re-trong-phat-trien-phan-mem-hien-dai/>.
- [53] Trung tâm Công nghệ và Đào tạo Robusta, “DevSecOps là gì và 3 bước để tích hợp bảo mật vào quy trình CI/CD của bạn,” 30 07 2025. [Trực tuyến]. Available: <https://robusta.vn/vi/goc-cong-nghe/devsecops-la-gi-va-3-buoc-de-tich-hop-bao-mat-vao-quy-trinh-cicd-cua-ban>.
- [54] T. PM, “Dependabot GitHub - Tìm hiểu tri thức,” VIBLO, 01 09 2024. [Trực tuyến]. Available: <https://viblo.asia/p/dependabot-github-tim-hieu-tri-thuc-AZoJdpyVY7>.
- [55] Thaycacac, “Dependabot trên GitHub: Hướng Dẫn Chi Tiết về Tự Động Hóa Quản Lý Dependencies và Bảo Mật Dự Án,” KUNGFUTECH, [Trực tuyến]. Available: <https://kungfutech.edu.vn/posts/dependabot-tren-github-huong-dan-chi-tiet-ve-tu-dong-hoa-quan-ly-dependencies-va-bao-mat-du-an>.
- [56] H. Mohayjei, A. Agaronian, E. Constantinou, N. Zannone và A. Serebrenik, “Securing dependencies: A comprehensive study of Dependabot’s impact on vulnerability mitigation,” 31 12 2025. [Trực tuyến]. Available: <https://link.springer.com/article/10.1007/s10664-025-10638-w>.
- [57] PRISMA CLOUD, “Policy-as-code for everyone,” [Trực tuyến]. Available: <https://www.checkov.io/>.

- [58] devoteam, “Trivy Security Scanner: Find Vulnerabilities Fast!,” [Trực tuyến]. Available: <https://www.devoteam.com/expert-view/shift-left-security-in-your-cloud-native-environment-with-trivy/>.
- [59] Viện FMIT, “Cosign là gì - Cosign là gì,” [Trực tuyến]. Available: <https://fmit.vn/tu-dien-quan-ly/cosign-la-gi>.
- [60] Sigstore, “Overview,” [Trực tuyến]. Available: <https://docs.sigstore.dev/cosign/signing/overview/>.
- [61] Chainguard, “An Introduction to Cosign,” [Trực tuyến]. Available: <https://edu.chainguard.dev/open-source/sigstore/cosign/an-introduction-to-cosign/>.
- [62] harness, “Securing CI/CD Images with Cosign and OPA,” 14 11 2023. [Trực tuyến]. Available: <https://www.harness.io/blog/securing-images-cosign-opa>.
- [63] Amazon Web Services, “Why AWS?,” [Trực tuyến]. Available: <https://aws.amazon.com/what-is-aws/>.
- [64] Trainocate, “AWS là gì? Tìm hiểu nhanh gọn về Amazon Web Services,” [Trực tuyến]. Available: <https://trainocate.com.vn/aws-la-gi-tim-hieu-nhanh-gon-ve-amazon-web-services-a115>.
- [65] FPT Aptech, “AWS là gì? Những thông tin chi tiết về AWS,” [Trực tuyến]. Available: <https://aptech.fpt.edu.vn/aws-la-gi.html>.
- [66] Amazon Web Services, “Amazon EKS features,” [Trực tuyến]. Available: <https://aws.amazon.com/eks/features/>.
- [67] F. Cipollone, “The Acceleration: How Supply Chain Attacks Went Industrial Across npm, PyPI, VS Code, and AI Agent Tooling,” Phoenix

- Security, [Trực tuyến]. Available: <https://phoenix.security/accelerating-supply-chain-attacks-npm-pypi-vsx-ai-enabled-2026/>.
- [68] S. D. G. & L. L. Hudon, “Combating Supply Chain Cyber Threats: Safeguarding Data and Protecting Digital Supply Chains in a Rapidly Evolving Cyber Landscape,” Foley & Lardner LLP, [Trực tuyến]. Available: <https://www.foley.com/insights/publications/2025/10/combating-supply-chain-cyber-threats-and-protecting-digital-supply-chains/>.
- [69] Ramaswamy Chandramouli (NIST), “Integrating Cybersecurity and Enterprise Risk Management,” National Institute of Standards and Technology (NIST), 2022. [Trực tuyến]. Available: <https://doi.org/10.6028/NIST.SP.800-204C>.
- [70] Cloud Security Alliance (CSA), “Top Threats to Cloud Computing Pandemic Eleven,” CSA Report, 2022. [Trực tuyến]. Available: <https://cloudsecurityalliance.org/artifacts/top-threats-to-cloud-computing-pandemic-eleven/>.
- [71] M. N. O. Cobo, A. S. C. Rivera và a. J. M. S. Perez, “DevSecOps: A Multivocal Literature Review,” *IEEE Access*, tập 8, p. 211246–211258, 2020.
- [72] “Tự quản lý Docker image tại chỗ có an toàn? Cách kiểm soát và mã hóa image qua Container Registry,” [Trực tuyến].
- [73] N. Chauhan, “How to Automate Vulnerability Scans with Trivy,” 24 11 2025. [Trực tuyến]. Available: <https://dev.to/addwebsolutionpvtltd/how-to-automate-vulnerability-scans-with-trivy-3b14>.
- [74] O. RACHIQ, “Integrating Trivy for Vulnerability Scanning in a DevSecOps Jenkins Pipeline.,” 20 09 2024. [Trực tuyến]. Available:

<https://skinnysyd.medium.com/integrating-trivy-for-vulnerability-scanning-in-a-devsecops-jenkins-pipeline-e57de5b27ada>.